Typed Capability Context for LLM Agents: Discovery, Progressive Disclosure, and Reactive Execution

Jorge Simao
EInnovator R&D – Center for the Sciences of Computation, Cognition, and Complexity

August 2026

Experimentally Frozen Editorially Consolidated Ready for External Review

Abstract

Large agents may register hundreds or thousands of tools and skills even though one decision typically needs only a few. Exposing every schema and instruction wastes active context; forcing an external router to choose exactly one removes the language model's ability to discriminate among plausible capabilities. We introduce a typed capability-context architecture that searches a large registry outside model context, presents a bounded palette of compact selection views, lets the model choose a stable capability identity, and then activates only that capability's complete record. Tool execution remains subject to a separate host authorization boundary.

Ordinary callable metadata supports useful registration: validation-frozen automatic fusion reaches 0.812 Top-1 on the zero-manual-metadata cohort, statistically unresolved from 0.806 for rich manual metadata. On five seeded 8,192-tool catalogs, diversity-preserving retrieval raises required-tool recall from 0.653 at $K = 8$ to 0.701 at $K = 32$ and 0.746 at $K = 48$. Candidate breadth has a model-side cost. At $K = 10$, a frozen Qwen3-0.6B chooser reaches 0.667 conditional accuracy when the target is present and 0.250 end-to-end accuracy; at $K = 32$, conditional and end-to-end accuracy fall to 0.400 and 0.150 on the same executable cohort. Deterministic selection-to-full disclosure saves 51.3% of tool capability tokens and active native K/V at $K = 32$; for 50 declarative skills, the saving is 84.3%. Large-catalog two-step workflows still fail when per-step target recall is only 0.250. The supported deployment pattern is thus broad external search, a small high-recall typed palette, exact full-record activation after choice, and independent authorization. Progressive disclosure makes broader palettes economical; it does not repair missing candidates, model confusion, or unsafe execution.

1 Introduction

An agent's logical capability environment can be much larger than the context needed for one decision. A software assistant may have thousands of APIs, data operations, command wrappers, and reusable procedures. Copying every complete definition into every request couples prompt and native key/value cost to the registry size rather than to the work at hand.

The opposite extreme is also brittle. An external router that exposes exactly one tool must solve the entire intent decision before the model sees any alternatives. A ranking mistake becomes unrecoverable, and ambiguity cannot be resolved by comparing plausible schemas. The practical systems problem is therefore not "all tools or one tool." It is how to produce a bounded, high-recall candidate palette that the model can still discriminate.

We add a second separation. The information needed to *choose* a capability is smaller than the information needed to *use* it. A tool can be selected from its name, typed signature, short

Capability	Compact selection view	Full use view
Tool	name, typed signature, short description, side-effect marker	complete provider schema, parameter descriptions, required/optional constraints, return schema, side effects
Skill	name, description, `when_to_use`	full instructions, constraints, ordered steps, examples, dependencies, references

Table 1: Selection and use are deterministic views of one typed identity. Neither view is produced by lossy summarization of the other.

description, and side-effect class, but valid invocation requires its complete schema and parameter constraints. A skill can be selected from its name, description, and applicability, while use requires full instructions and constraints. These are deterministic named views of one typed record, not summaries generated from one another.

The resulting architecture has six stages:

1. normalize callables and declarative skills into typed, versioned records;
2. search the large registry with model-independent sparse, semantic, and hybrid channels;
3. expose a bounded palette of compact selection views;
4. let the model choose a stable record identity;
5. activate only the selected full record without semantic rediscovery;
6. authorize and execute tools at the host boundary, or use full skill instructions to guide generation.

This paper makes four claims. First, useful registration does not require extensive manual semantic annotation. Second, large registries should be searched outside active model context and reduced to bounded palettes. Third, typed progressive disclosure reduces active token and K/V cost while preserving one complete selected record. Fourth, reactive execution remains bounded by both candidate recall and model discrimination as catalog size and K increase. Detailed controls and the complete experimental chronology are retained in the appendices.

2 Typed Capability Context

A capability record has a stable URI, type, namespace, version, source fingerprint, aliases, provenance, access scope, and named views. Identity is distinct from the text used for discovery, the encoded representation cached by a model runtime, and current attention visibility. Updating source or model state therefore invalidates the relevant fingerprint without changing the conceptual resource namespace.

Figure 1 shows the system boundary. Discovery may scan a large registry, but only the chosen palette enters model-visible context. A model selection identifies a record; it does not grant permission to call it. Tools cross a separate host authorization and validation boundary. Skills remain declarative text and do not acquire execution authority.

The design distinguishes six costs that are often conflated:

- **backing-store bytes**: source and normalized record payloads;
- **index bytes**: sparse, dense, and provenance-bearing retrieval structures;

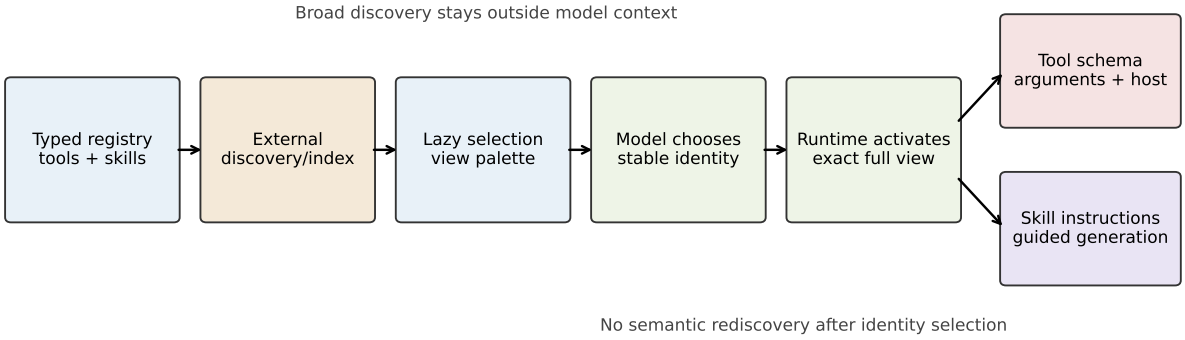

Figure 1: One capability-context architecture. Registration establishes stable typed identity; external discovery produces a bounded compact palette; the model chooses an identity; the runtime activates exactly one full record; tool execution remains host-authorized.

- **resident encoded bytes**: cached representations that may be reused but are not currently visible;
- **transferred K/V**: encoded state moved across a process or device boundary;
- **active K/V**: state admitted to the current attention workspace;
- **prompt tokens**: serialized text when native activation is not available.

Savings in one category do not imply savings in every other category. The main tables report active token and K/V accounting; local cache and index costs are reported separately in the appendices.

2.1 Identity, state, and authorization invariants

The stable URI is the join key across registration, retrieval, cache lookup, model choice, and execution. A representative identity is `!!ref:tool:calendar:create_event:v3!!`; a skill uses the same scheme with a different type component. The text shown during selection is never used as the execution identifier. This prevents a shortened display description, an alias collision, or a later metadata revision from silently changing which binding is invoked.

Three invariants follow. First, every visible view retains type, URI, version, and source provenance. Second, a transition from selection to full use can only expand the same identity; it cannot substitute the nearest semantic neighbor. Third, access filtering precedes ranking. Tenant scope, revocation, and an optional URI allowlist therefore constrain the search domain before a candidate receives a score. The resolver returns a trace containing channel provenance, confidence, fallback count, and the final `select`/`ask`/`abstain` decision, but it never executes a tool.

Cache identity is correspondingly stricter than record identity. A reusable encoding fingerprint includes source version, model, tokenizer, routing configuration, and position-sensitive encoding state. A stale cached view can be invalidated without inventing a new logical capability; a revoked record can remain in backing storage without remaining eligible for discovery.

2.2 Discovery policy versus disclosure policy

The SDK represents retrieval and model visibility as separate policies. A discovery policy determines how the registry is searched and how confidence is calibrated. A disclosure policy determines how many records are visible, which view each record exposes, and whether destructive or restricted candidates may appear. This separation matters operationally: broad search can increase recall without forcing every scored record into the model context, while an explicit URI can still be presented with a comparison palette when the caller requests one.

Resolve mode seeks one identity, $q \mapsto r^*$. Explore mode seeks a bounded set, $(q, state) \mapsto C_K$, and must be evaluated with recall, useful precision, unsafe exposure, and downstream choice. Neither mode changes the authorization boundary shown in Figure 1.

2.3 End-to-end runtime contract

The architecture can be implemented without coupling the retrieval index to a particular model provider. Registration emits immutable record versions and selection/full serializers. Discovery consumes only indexable record metadata and returns ordered URIs with scores and provenance. The model adapter consumes selection views and returns one URI. The record store resolves that URI to the full view. Finally, the host validates a proposed tool call against the full schema and current policy. A skill ends at guided generation rather than tool dispatch.

For one request, the contract is:

1. filter the registry by tenant, revocation, type, and caller allowlist;
2. retrieve and calibrate a candidate set outside model context;
3. serialize at most K compact views, retaining every stable URI;
4. parse one model-selected URI and reject identities outside the palette;
5. activate that identity's complete current full view by exact lookup;
6. validate arguments and host policy before any side effect.

The discovery trace and the activation trace therefore meet at the URI but remain separate audit objects. A resolver can be replaced, an index can be rebuilt, or a model adapter can change without weakening record atomicity or authorization. Conversely, a high retrieval score cannot bypass palette membership, exact activation, schema validation, or host permission.

The local implementation uses lazy view objects and a cache keyed by the encoding fingerprint described above. This makes registration inexpensive and permits warm full-record reactivation. It does not imply that an opaque remote API can alter an already-running attention state; such an adapter may serialize the same two phases as separate requests. The portable claim is the typed contract and absence of semantic rediscovery after choice, not a universal provider-specific cache operation.

3 Automatic Capability Registration

The public SDK accepts a complete `ToolRecord`, an ordinary Python callable, a declarative `Skill`, or a parent directory containing skill folders. Callable normalization extracts name, signature, annotations, docstrings, parameter descriptions, required/optional structure, return type, and side-effect metadata. Deterministic enrichers derive operation/object terms, generic synonyms, inferred

Registration surface	Top-1	Recall@3	Interpretation
Raw callable text	0.403	—	name/signature/docstring only
Automatic generic synonyms	0.750	—	largest isolated gain
Automatic fused metadata	0.812	0.938	validation-frozen zero-manual policy
Rich manual metadata	0.806	0.910	authored comparison ceiling

Table 2: Automatic registration on the frozen zero-manual-metadata cohort. The automatic/manual Top-1 difference is statistically unresolved on this cohort; detailed ablations are in the appendices.

concepts, and compact embedding text. The full schema remains authoritative even when a compact view is used for selection.

Skills use the same identity and discovery substrate but a different full payload. Common OpenAI- and Anthropic-style `SKILL.md` folders are normalized into provider-neutral declarative records. Ambiguous layouts are rejected. Script and asset paths are recorded for provenance but are not executed by this paper's runtime.

3.1 Deterministic registration pipeline

Callable registration begins from information already required to invoke the function: its qualified name, Python signature, type annotations, defaults, and docstring. The normalizer converts this material into a provider-neutral parameter schema and stores the original callable as a host binding. Metadata enrichment then derives compact retrieval fields in four deterministic steps: identifier splitting and normalization; operation/object extraction; generic synonym expansion; and typed concept/tag inference. Compact embedding text is constructed from those fields, while the complete generated schema remains the only full tool view.

This order avoids a circular evaluation. Retrieval metadata is generated from the callable and fixed rules, not from test-request labels or target IDs. The automatic and manual conditions share catalog identities, train/validation/ test partitions, queries, and frozen discovery policy. Only the registration surface changes. Validation chooses policy settings; test identities are used once for the reported comparison. The automatic condition is therefore a zero-manual-semantic-metadata condition, not a zero-documentation condition.

Skill normalization follows the same discipline. Front matter supplies name, description, and applicability; the body supplies full instructions. Folder layout and optional references contribute provenance but not hidden execution authority. The loader accepts a list of explicit `Skill` objects or a parent directory, discovers supported child folders, assigns stable URIs, and rejects duplicate or ambiguous identities. Provider-specific syntax is confined to loading; retrieval and progressive disclosure operate on the normalized record.

The automatic-registration comparison fixes train/validation/test identities and retrieval policy before evaluation. Table 2 separates raw callable text, the largest isolated automatic enrichment, the complete automatic policy, and the manually authored ceiling.

Automatic fusion reaches 0.812 Top-1 versus 0.806 for manual metadata. This does not show that documentation quality is irrelevant: removing docstrings reduces Recall@3 to 0.521. It shows that useful typed registration can begin from ordinary callable metadata rather than a separate hand-authored semantic catalog.

Record atomicity is part of registration correctness. When one complete tool record is selected, execution acceptance is 1.000. Generic internal chunk routing accepts only 0.150 and omits required

arguments in 0.850 of cases. The runtime therefore treats an externally selected tool schema as an authoritative atomic unit rather than as arbitrary text fragments.

The ablation identifies where registration quality comes from. Raw callable text is usable but incomplete. Generic synonyms provide the largest isolated gain, and the full automatic fusion combines complementary evidence without requiring a second catalog written by hand. The docstring-removal result shows the boundary: automatic extraction can remove duplicated annotation work, but it cannot recover semantics that the software interface never states.

4 Large-Catalog Discovery

Discovery and disclosure are independent controls. Discovery searches explicit URIs, lexical tokens and aliases, indexed BM25 evidence, typed operation/object concepts, compact embeddings, or fused combinations. Disclosure decides which retrieved records become model-visible. A policy can therefore use broad external search while enforcing a small compact palette.

4.1 Channels and decision policy

The resolver exposes six channel families. Exact URI lookup is deterministic after access checks. Lexical search uses normalized names, aliases, descriptions, and token evidence. Persistent postings and BM25 narrow a large catalog before scoring. Typed dictionaries preserve provenance for operation, object, and family concepts. Compact local embeddings provide paraphrase evidence without calling the generation model. Fusion combines ranked evidence; adaptive mode starts with cheaper high-precision channels and escalates only when calibrated confidence is insufficient.

For candidate r_i, confidence approximates

$$c_i = P(r_i \text{ is intended} \mid q, \mathbf{s}_i, p_i),$$

where $\mathbf{s}_i$ contains channel scores and p_i records their provenance. High confidence permits selection, intermediate confidence can trigger another channel or clarification, and low confidence abstains. A positive request is operationally correct only if the right resource is ranked first and the decision is `select`; an ambiguous or missing request is correct only if the policy asks or abstains. This prevents a latent ranker from receiving deployment credit for a candidate it should not safely expose.

Fusion and diversity union serve different roles. Frozen fusion produces one calibrated ranking and remains the SDK default. Diversity union takes bounded contributions from complementary channels, deduplicates identities, and preserves disagreement that a single fused score can suppress. It is an experimental high-recall palette generator. Raw union is retained only as a diagnostic because candidate coverage alone does not control irrelevant or unsafe exposure.

4.2 Evaluation cohorts

The semantic-hard cohort fixes 144 held-out requests across canonical, paraphrastic, colloquial, implicit, contextual, and multilingual forms. It tests whether a resolver can find the same capability when surface overlap is weak. Validation freezes channel thresholds and fusion settings before test evaluation. The staged resolver reports both coverage and selective accuracy, because abstaining on every hard request would otherwise appear safe.

The scaling cohort preserves those target identities while adding callable-derived, semantically confusable distractors at catalog sizes 32, 128, 512, 2,048, and 8,192. Five catalog seeds vary distractor identity and ordering. Candidate budgets span $K = 1$ through 48. The study records Recall@K, useful precision, unsafe exposure, latency, index memory, and active-context cost. Model-choice

experiments are more expensive and use a separately identified eight-request by five-seed cohort; their denominator is never substituted for the 720-row retrieval denominator.

The semantic-hard benchmark contains 144 frozen test requests spanning canonical, paraphrastic, colloquial, implicit, contextual, and multilingual forms. BM25 obtains .347 Top-1; typed tags obtain .743; and the dictionary/embedding hybrid obtains .729. A validation-calibrated staged resolver selects on .597 of requests at .907 selective accuracy. Confidence therefore controls action coverage rather than merely decorating a ranking.

The scaling study adds callable-derived, semantically confusable distractors through 8,192 tools under five catalog seeds. Frozen hybrid Top-1 falls from 0.787 at 32 tools to 0.338 at 8,192; lexical Top-1 at 8,192 is 0.451. At $K = 10$, raw union raises required- tool recall to 0.662, but useful precision is only 0.078 and unsafe exposure is 0.110. Broad union is therefore a candidate generator, not a default disclosure policy.

The supported resolver uses model-independent indexed lexical evidence, provenance-bearing dictionaries/tags, compact embedding fallback, and validation-calibrated escalation. Frozen native Q/K and transferred low-rank projections obtain only .049–.063 Top-1 on the same semantic-hard test set. That negative control defines the current deployment boundary; layer, reducer, and projection diagnostics remain in the appendix.

The scaling result is not a claim that lexical evidence universally dominates semantic evidence. It shows a changing channel geometry: exact lexical cues remain valuable as confusable descriptions accumulate, while paraphrastic queries still require semantic or typed evidence. A production resolver should therefore index complementary channels and calibrate their union, rather than replace one exhaustive Python scorer with another. Cold/warm timing, posting memory, and refresh costs are reported in the appendix because they depend on the local implementation and catalog representation.

5 Candidate Palettes and Model Choice

The candidate budget K trades retrieval recall against model discrimination. These are different random variables:

$$R_K = P(r^* \in C_K), \tag{1}$$

$$A_K = P(\widehat{r} = r^* \mid r^* \in C_K), \tag{2}$$

$$E_K = P(\widehat{r} = r^*) = R_K A_K. \tag{3}$$

A retrieval-only Recall@K curve must not be presented as target presence in a different executable cohort. We therefore report the five-seed 8,192-tool retrieval frontier and the smaller frozen-model choice cohort separately.

5.1 Matched palette protocol

Each model-facing row serializes the same compact fields in a stable order and asks frozen Qwen3-0.6B to return one bound capability identity. The target may be absent; such a row is a retrieval failure for end-to-end choice, not a model error in conditional choice. When the target is present, distractor count and composition define the discrimination problem. Seeds are paired across policy conditions so an end-to-end difference is not attributed to a different catalog draw.

The three metrics answer different deployment questions. R_K asks whether external discovery preserved an actionable option. A_K asks whether the model can distinguish that option from the visible alternatives. E_K asks whether the combined pipeline selects it. Accepted-call rate adds

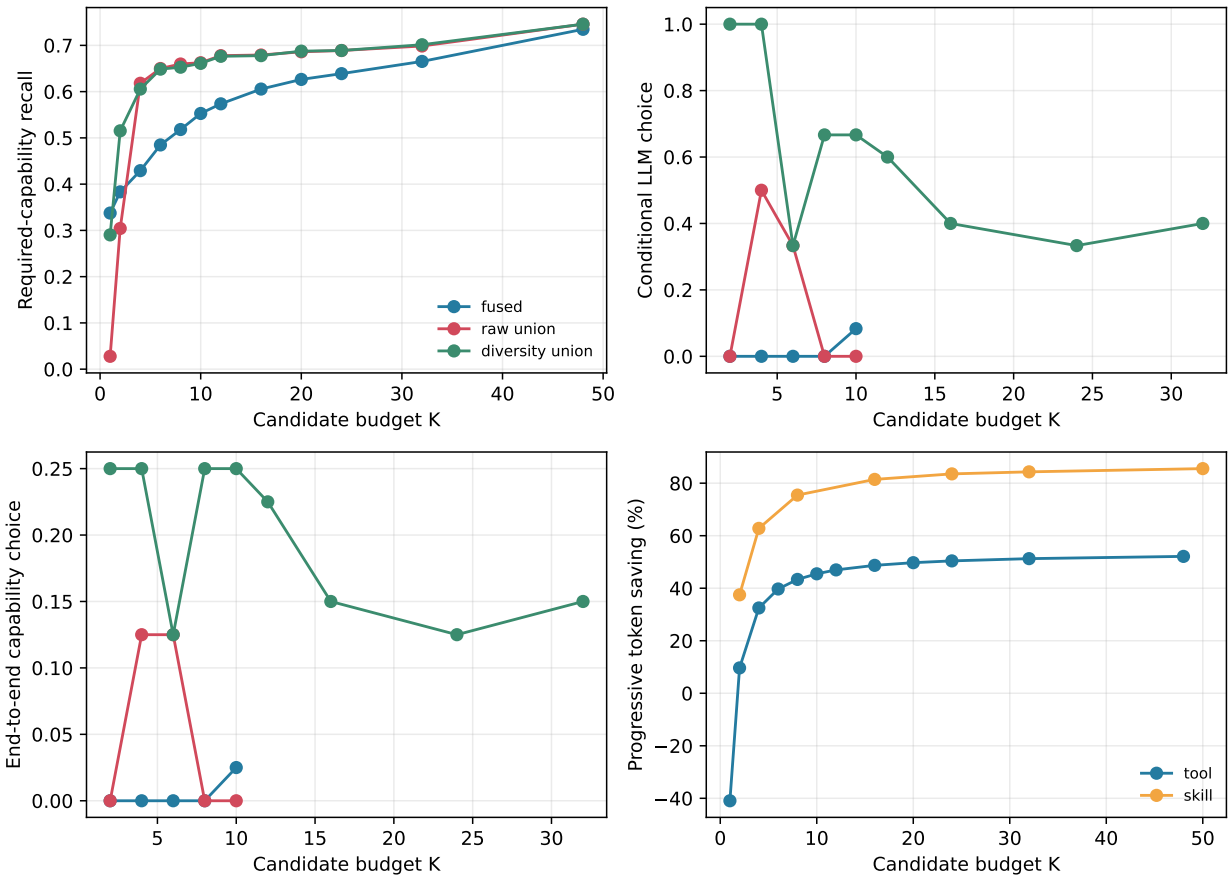

Figure 2: The K frontier. Panels separate required-tool recall, conditional model choice, end-to-end choice, and progressive-disclosure savings. Retrieval and executable cohorts are labeled separately; unmeasured policy points are not interpolated.

one more gate: after selection, the model must construct arguments accepted by the authoritative schema and host validator. Reporting only Recall@K would hide the latter three failure surfaces.

Figure 2 is the central operating result. Diversity- preserving recall rises from 0.653 at $K = 8$ to 0.678 at $K = 16$, 0.701 at $K = 32$, and 0.746 at $K = 48$. The gain from quadrupling $K = 8$ to $K = 32$ is 4.9 percentage points, while useful precision falls. Candidate breadth is useful, but its return diminishes.

At $K = 10$, diversity union places the target in 15 of 40 executable palettes, chooses it in 10 of those 15, and reaches 0.667 conditional and 0.250 end-to-end choice. Frozen fusion reaches 0.300 target recall, 0.083 conditional choice, and 0.025 end-to-end choice. The paired end-to-end effect favoring diversity union is exact $p = 0.0117$ on this fixed cohort.

More slots do not continue that improvement. Table 3 holds the eight requests and five catalog seeds fixed. Target presence remains 15/40 from $K = 10$ through $K = 32$, while conditional choice falls from 0.667 to 0.400 and end-to-end choice from 0.250 to 0.150. Larger compact palettes are affordable, but they are not free of decision interference.

The practical operating point is consequently policy- and model-dependent. At the retrieval scale, increasing K through 48 continues to recover targets, but gradually. On the executable cohort, $K = 10$ is the best measured diversity point because additional slots add no targets and

K	Rows	Target present	Recall	Conditional	End-to-end
10	40	15	0.375	0.667	0.250
12	40	15	0.375	0.600	0.225
16	40	15	0.375	0.400	0.150
24	40	15	0.375	0.333	0.125
32	40	15	0.375	0.400	0.150

Table 3: Measured diversity-union model choice on the fixed 40-row executable cohort. Conditional accuracy excludes target-absent palettes; end-to-end accuracy counts them as failures.

reduce conditional choice. The paper does not claim that ten candidates is universal: a larger model, a different tokenizer, or a less confusable catalog may move the optimum. It claims that K must be calibrated jointly against retrieval and downstream choice, not selected from retrieval recall alone.

Static capability neighborhoods provide a useful negative control. A typed graph built from categories, objects, operations, curated tags, description keywords, and producer–consumer schema edges attains high required-tool coverage. Yet the frozen workflow model completes none of 15 tasks under broad static disclosure. Coverage without a query-conditioned decision point can increase visible confusion, so capability-graph expansion is not part of the recommended default path.

6 Typed Progressive Disclosure

After candidate retrieval, the runtime activates compact selection views lazily. The model chooses one stable URI from that palette. The local transition

$$(C_K, r^*) \longmapsto \text{activate\_full}(r^*)$$

performs no BM25, embedding, fusion, or union query. The already-known identity resolves directly to its complete tool schema or skill instructions. Cached full encodings can be reactivated without recomputation. An opaque remote provider may still require another model request; the narrower guarantee is that the runtime does not rerun semantic discovery.

6.1 Deterministic views and lifecycle

Progressive disclosure is not free-form summarization. The selection serializer projects a fixed field allowlist from the normalized record. For tools, it includes identity, name, typed signature, short description, and the available side-effect marker. For skills, it includes identity, name, description, and `when_to_use`. The full serializer projects the complete validated tool schema or declarative instruction body. Unknown fields cannot leak into a selection view, and required full fields cannot be dropped by a generative compressor.

The lifecycle separates source residency from model visibility. Registration can remain metadata-only. A first query encodes or serializes the selected compact views. Model choice fixes the URI. Exact activation then looks up that URI and admits its cached full encoding, or encodes it once on a cold path. Losing candidates are removed. This gives the model comparison breadth at choice time without paying the full-use cost for every candidate.

The state machine is

$$\textsc{Unencoded} \to \textsc{SelectionEncoded} \to \textsc{ActiveSelection} \to \textsc{FullEncoded} \to \textsc{ActiveFull}.$$

Type	K	Full-all tok.	Progressive tok.	Saving	Full KV MiB	Prog. KV MiB
Tool	8	2,564	1,453	43.4%	280.5	158.9
Tool	32	10,455	5,095	51.3%	1,143.5	557.3
Skill	8	6,866	1,674	75.5%	751.0	183.1
Skill	32	26,617	4,170	84.3%	2,911.3	456.1

Table 4: Selection views plus one selected full record versus exposing every candidate in full. K/V values are active/attention-visible accounting, not backing-store, index, or resident-cache bytes.

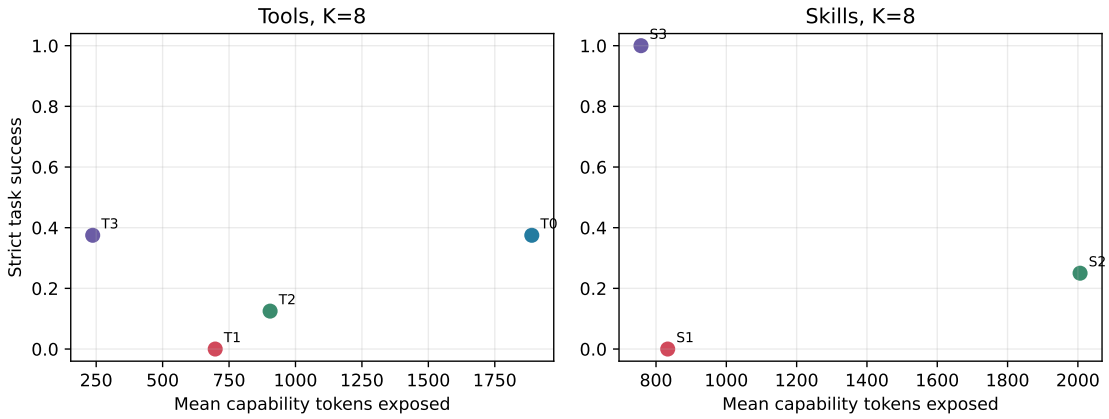

Figure 3: Typed selection-to-full quality-cost frontier. Compact views lower capability exposure; one complete authoritative record is activated only after choice. Full per-condition tables remain in the appendix.

Resident cached bytes and attention-visible active bytes are accounted separately. Table 4 gives representative means using the frozen Qwen accounting value of 114,688 native-K/V bytes per active token.

For palette C_K and selected record r^*, the serialized capability-token costs are

$$T_{\text{full}}(C_K) = \sum_{r \in C_K} T(F(r)), \tag{4}$$

$$T_{\text{prog}}(C_K, r^*) = \sum_{r \in C_K} T(S(r)) + T(F(r^*)), \tag{5}$$

where S and F are deterministic selection and full views. The reported saving is $1 - T_{\text{prog}}/T_{\text{full}}$. Native-K/V accounting applies the model-specific per-token value only to attention-visible views. Backing records and cached inactive encodings are reported separately rather than counted as memory eliminated by the method.

The 50-skill benchmark tests whether the abstraction generalizes beyond executable APIs. Validation-frozen fusion reaches 0.94 Recall@4, 0.96 at $K = 8$, and 1.00 at $K = 16$ on 50 name-blind held-out requests. The median skill selection/full token ratio is 0.150. At $K = 8$, selection plus one complete skill uses 0.245 of full-all tokens; at $K = 32$, it uses 0.157.

The benchmark contains 50 confusable declarative skills and 100 name-blind requests split evenly between validation and test. Requests include canonical, paraphrastic, colloquial, goal-oriented, contextual, and multilingual forms; instruction bodies span short, medium, and long buckets. Tool and skill retrieval use the same resolver interface and stable identity rules, while

Metric	Tools	Skills
Logical catalog size	8,192	50
Held-out discovery queries	5×144	50
Recall at $K = 8$	0.653	0.960
Recall at $K = 32$	0.701	1.000
Conditional choice at reported gate	0.667 ($K = 10$)	0.500 ($K = 8$)
Progressive strict success	0.250 ($K = 8$)	0.250 ($K = 8$)
Progressive token saving	0.513 ($K = 32$)	0.843 ($K = 32$)
Active-K/V saving	0.513 ($K = 32$)	0.843 ($K = 32$)

Table 5: Tools and skills under one typed-capability architecture. Discovery rows are larger than model-use rows. Tool recall uses diversity union; skill recall uses validation-frozen fusion. Conditional choice is evaluated only when the target is present.

their full-view serializers and use-time checks remain type-specific.

The model-use result is more modest. At $K = 8$, no full-all skill prompt fits the measured 4 GB runtime, while all selection-to-full prompts fit. The frozen model chooses and applies the target skill in 1/4 cases; oracle-full succeeds in 4/4. The compact representation makes the condition executable and preserves a full-instruction ceiling, but it does not solve model choice.

An eight-request mixed 8,192-tool/50-skill check reaches 0.75 retrieval recall for both target types at $K = 8$. Exact choice is 0.50 for tools and 0.25 for skills. At $K = 32$, every tool target is present but exact tool choice falls to 0.25. Skills therefore support the shared typed representation; they do not justify a second discovery architecture or a claim of broad procedural competence.

The shared table should not be read as a head-to-head benchmark. Tool catalogs are much larger and use five distractor seeds; skill discovery has fewer identities and a different use-time objective. Its purpose is architectural: the same registration, bounded palette, stable URI, lazy activation, and cost accounting apply to both record types without flattening either into an untyped text chunk.

6.2 What progressive disclosure does and does not preserve

Four conditions isolate the mechanism. *Full-all* exposes every candidate in full. *Selection-only* tests whether compact views can support choice but intentionally withholds use details. *Selection-to-full* expands the chosen identity before use. *Oracle-full* supplies the correct complete record and estimates the ceiling once retrieval and choice are removed. The gap between selection-only and selection-to-full tests the value of exact expansion; the gap from selection-to-full to oracle-full localizes remaining retrieval or model-choice error.

The atomicity control explains why exact full activation matters. The selected record preserves parameter names, required/optional constraints, return schema, and side-effect metadata as one unit. Routing generic internal chunks may retrieve locally relevant text while omitting the field that makes the call valid. Progressive disclosure reduces the number of full records; it does not fragment the one record that survives.

7 Reactive Execution and Scaling Limits

Reactive execution refreshes discovery after each observation instead of placing an entire anticipated plan in context. On the small 18-tool cohort, the frozen model completes 10/15 three-to-five-step workflows with reactive just-in-time disclosure, compared with 3/15 when all required tools are

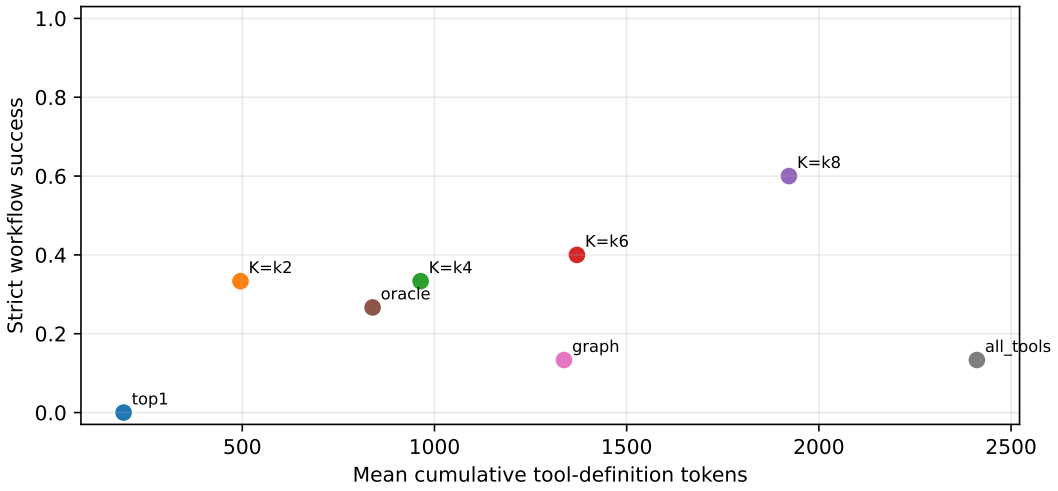

Figure 4: Reactive execution frontier on the frozen workflow controls. Candidate breadth can reduce disclosure cost only after each required identity is retrieved. Large-catalog JIT remains recall-limited; detailed traces and policy variants are in the appendix.

shown statically and 0/15 without refresh. The mechanism can therefore help when each step's discovery and choice are reliable.

7.1 Reactive protocol

At each step, the host serializes the current request and prior observation, runs the frozen resolver, exposes a bounded selection palette, activates the chosen full tool record, validates the proposed arguments, executes the local binding, and appends a typed observation. The next discovery query is formed from this updated state. No workflow receives the target sequence as a plan, and a failed or unauthorized call cannot be repaired by silently dispatching an oracle tool.

The comparison isolates timing of disclosure. Static-required exposes the known required set before generation; reactive JIT discovers from the current state; no-refresh fixes the initial palette. The small-catalog result shows that a fresh observation can make the next capability easier to identify. It does not establish that repeated retrieval scales automatically to thousands of confusable tools.

The large-catalog result is the limiting case. All ten tested two-step workflows fail because per-step target recall is only 0.250. For a workflow of length H, the expression

$$P(\text{all required targets retrieved}) \approx R^H$$

is a retrieval-only diagnostic under an independence approximation, not a measured law. It illustrates why even moderate per-step miss rates compound; actual steps may be correlated and model choice adds another failure term.

A useful failure decomposition for step h is

$$P(\text{accepted}_h) = P(r_h^* \in C_K)\, P(\widehat{r}_h = r_h^* \mid r_h^* \in C_K)\, P(\text{valid call}_h \mid \widehat{r}_h = r_h^*).$$

Each factor is conditional on the accumulated state. Multiplying a single average recall as R^H deliberately ignores that dependence; it is included only to show why a per-step recall of 0.250

leaves little room for a two-step workflow. The measured outcome, rather than the approximation, is that all ten large-catalog workflows fail.

Two negative controls narrow the recommended path. Static capability-graph disclosure reaches high required-tool coverage but 0/15 task success, so broad speculative neighborhoods are not a substitute for reactive choice. Frozen native Q/K does not provide useful semantic-hard discovery signal in the tested setting. Fusion remains the conservative SDK default; diversity union remains an experimental high-recall candidate generator.

The security boundary is unchanged by any discovery policy. Candidate visibility and model selection do not authorize execution. The host validates the selected full schema, arguments, tenant scope, revocation state, and side-effect permission before dispatch.

The result yields a conservative deployment rule. Reactive refresh is useful where per-step resolver recall and conditional choice have been measured on the relevant catalog. It should degrade to clarification, abstention, or an operator-approved broader search when confidence is low. Progressive disclosure can make that broader palette cheaper, but cannot turn an absent target into a present one or make a confusing palette discriminable.

8 Related Work

8.1 Tool retrieval and large catalogs

Gorilla combines API-call generation with retrieved documentation and studies adaptation to changing APIs [2]. ToolLLM introduces ToolBench, instruction tuning over more than 16,000 APIs, a neural API retriever, and multi-step search [3]. ToolRet later isolates retrieval over heterogeneous large tool collections and shows that general retrievers need not transfer cleanly to capability search [8]. Paper 6.5 separates candidate retrieval, conditional model choice, accepted call construction, and host execution so a high Recall@K result cannot stand in for downstream success.

8.2 Tool and function calling

Toolformer learns when and how to insert calls during language-model training [1]. Common function-calling paths expose either all schemas or a router-selected subset before the model constructs arguments. Our runtime adds a distinct compact palette: the model first chooses among typed selection views, then receives one exact complete schema. Record atomicity and host authorization remain explicit.

8.3 Progressive disclosure and context management

Prompt-compression systems such as LLMLingua shorten model-visible text under a budget [9]. Headroom's public Compress–Cache–Retrieve architecture retains compressed originals and permits tool-triggered or proactive recovery [10]. Paper 6.5 addresses capability definitions rather than large result payloads. Its selection and full views are deterministic typed projections, not adaptive summaries. Reversible result compression, multiple address views, and stateful cursors belong to Paper 7 and are outside this paper's scope.

8.4 Skills and procedural capabilities

Public agent-skill formats package trigger metadata, instructions, and optional resources in addressable folders [7]. We normalize common declarative layouts into the same typed identity/discovery

substrate as tools, while preserving different full payloads and excluding scripts. The skill benchmark is evidence for representation generality, not a claim that tool execution and procedural guidance are behaviorally identical.

8.5 Hybrid and model-native retrieval

BM25 supplies sparse lexical evidence, dense passage retrieval supplies complementary learned evidence, and reciprocal rank fusion combines ranked lists without equating their scores [11, 4, 12]. Query-aware KV systems such as Quest select cache pages for an already formed attention query [6]. Paper 6.5 operates before full capability materialization. Its frozen native-Q/K result is a bounded negative control, not a general argument against trained model-native routing.

9 Limitations

The generation model is frozen Qwen3-0.6B. Conditional choice given target presence is model-dependent, and the K that works best here need not transfer to a larger model or another tokenizer. Retrieval cohorts are substantially larger than model-choice, mixed-palette, and workflow cohorts. Exact paired tests are retained where justified, but small raw counts remain small raw counts.

Semantic-hard queries and confusable catalogs are authored or generated rather than sampled from production traffic. Capability domains are limited. The mixed tool/skill study has eight held-out requests, workflows are short, and large-catalog discovery remains imperfect. Larger K can improve recall while increasing model confusion.

The reference resolver still contains exhaustive Python components. Indexed latency is measured locally, not under a production remote-serving topology. The paper does not provide a comprehensive adversarial security evaluation, distributed cache policy, cross-tenant residency study, or remote-provider request/TTFT analysis. Tool execution uses an in-memory harness.

Skills are declarative only. Scripted skills, executable assets, mutation, session inheritance, and long-horizon procedural evaluation are excluded. Large typed result records, reversible compression, adaptive materialization, and analytics cursors are Paper 7 mechanisms and are intentionally out of scope. Progressive disclosure here concerns capability definitions only.

10 Conclusion

A practical large-capability agent should:
1. normalize tools and skills into stable typed records;
2. search the large registry outside active model context;
3. disclose a bounded, compact, high-recall palette;
4. let the model choose one stable capability identity;
5. activate only that identity's complete full record;
6. authorize and execute tools separately from model choice;
7. refresh reactively only where per-step discovery quality supports it.

Automatic registration makes the architecture usable without a second manual catalog. Large-catalog retrieval makes broad search possible. Typed progressive disclosure makes bounded candidate breadth economical. The measured boundary is equally important: progressive disclosure reduces context cost; it does not solve missing candidates, model confusion, or authorization.

A M0 Experimental Design

A.1 Opaque catalogs

The generator in `src/data/agent_resources.py` creates identifiers such as `tool_zaf_193`. Each resource combines held-out action, object, and family concepts with aliases, a schema, a version, and a side-effect class. The opaque names prevent pretrained knowledge from solving the benchmark. Catalogs contain 8, 32, 128, 512, 2,048, and 8,192 resources.

Each held-out identity produces six positive requests: explicit URI, exact name, alias, typo, semantic paraphrase, and descriptive reference. Each split also contains one ambiguous and one nonexistent request. Validation identities calibrate thresholds; test identities are disjoint. At sizes 32–8,192, each policy is evaluated on 540 positive and 10 negative test requests across seeds $\{11, 23, 37, 53, 71\}$. The 8-resource condition uses 180 positive and 10 negative requests. The complete artifact contains 31,680 validation and test policy traces.

A.2 Policies and metrics

We compare five fixed policies (explicit, token, index, semantic, and hybrid), unguided `auto`, a request-level hint, and adaptive fallback. The oracle is the least-cost fixed policy that ranks the target first on that query. Reported metrics include top-1 accuracy, selection coverage, selective and actionable outcome accuracy, safe nonaction and false-action rates, retrieval stages, fallback count, Brier score, expected calibration error (ECE), quality regret, and policy-cost regret. Confidence thresholds are fitted only on the validation split. Intervals are percentile bootstrap intervals over five seed means; deterministic outcomes can consequently have zero-width intervals.

A.3 Systems accounting

Cold index construction and a full immutable rebuild after mutating one percent of resource versions are timed separately. Warm component latency is the mean of repeated CPU Python calls. Policy-path latency is reconstructed by summing the measured components actually executed by each trace. These timings exclude tokenization, a model forward pass, native-K/V encoding or transfer, attention, and tool execution. They characterize this reference implementation, not an optimized serving system.

B Results

B.1 Selection scales, but policies differ operationally

Table 6 reports the largest catalog. Hinted and adaptive policies select every positive target and safely reject every negative request. Their equal quality hides a meaningful systems difference: hints reduce the mean path from 2.39 stages to 1.08 and the measured component estimate from 1,084 to 691 ms per request. The fixed hybrid ranker also attains 100% top-1, but calibration leaves 1.7% of positive requests unselected. Its exhaustive 1,405 ms component cost and 4.17 cost-regret units make it a poor default.

Table 6: Held-out M0 results at 8,192 resources. Top-1 is computed on 540 positive requests; outcome includes ten ambiguous/nonexistent controls. Time is reconstructed CPU Python discovery-component time.

Policy	Top-1	Coverage	Outcome	Safe neg.	Stages	ms/query
Explicit	.167	.167	.182	1.000	1.00	5
Token	.713	.537	.545	1.000	1.00	1,174
Index	.833	.000	.018	1.000	1.00	27
Semantic	.333	.000	.018	1.000	1.00	339
Hybrid	1.000	.983	.984	1.000	1.00	1,405
Auto	.833	.000	.018	1.000	1.00	24
User hint	**1.000**	**1.000**	**1.000**	**1.000**	1.08	691
Adaptive	**1.000**	**1.000**	**1.000**	**1.000**	2.39	1,084

The unguided automatic policy exposes why ranking is insufficient. It ranks explicit, name, alias, typo, and description requests correctly, misses the semantic-paraphrase stratum, and obtains .833 top-1. Calibration nevertheless causes it to ask on every positive request, so its actionable outcome accuracy is only $2/110 = .018$ per seed: the two safe negative decisions. Fixed index has the same operational pattern. Conversely, fixed token acts selectively and is always right when it acts, but covers only .537 of positive requests.

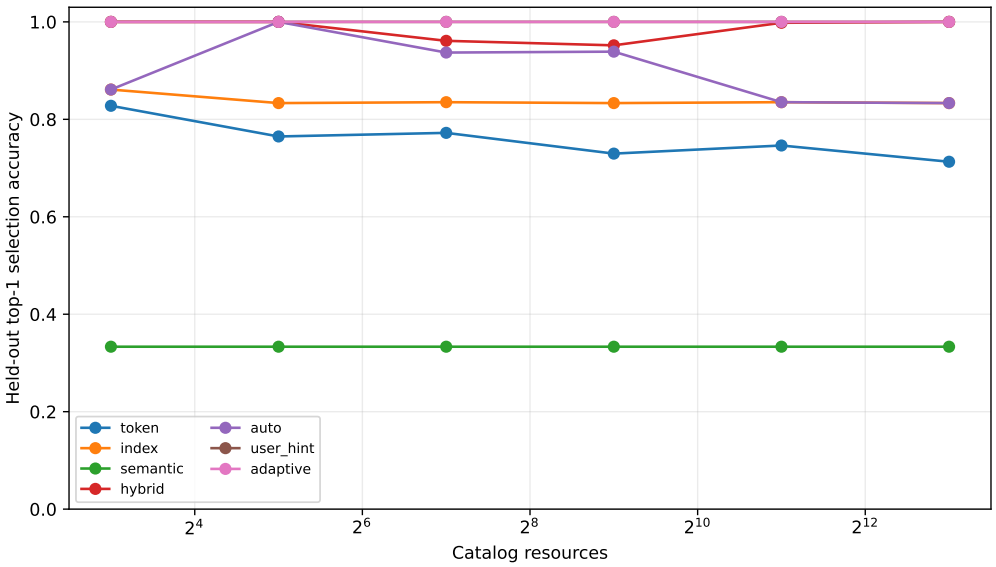

Figure 5: Positive-request top-1 accuracy as catalog size increases. Ranking alone does not encode whether confidence permits selection.

Figure 5 shows that hints and adaptive fallback maintain 1.0 top-1 at every tested size. The result is not uniformly safe at the smallest size: in the 8-resource condition each policy attains 1.0 positive top-1, but hinted and adaptive routing each falsely select the ambiguous held-out case in all five seeds. Their overall outcome accuracy is .974 and safe-nonaction rate is .5. From 32 resources onward, both negative cases are handled correctly in all seeds. This inversion is a calibration artifact of the controlled score distribution and cautions against treating larger catalogs as automatically harder along every axis.

B.2 No fixed channel is the oracle everywhere

At 8,192 resources, the least-cost correct oracle chooses explicit lookup for 16.7% of positive requests, indexed lookup for 66.7%, and semantic lookup for 16.7%. Neither token nor hybrid is ever the least-cost correct choice. User hints have zero quality regret and 0.083 cost-regret units, compared with zero quality regret and 0.5 cost regret for adaptive fallback. This supports a narrow claim: when callers know the reference form, passing that information avoids discovery work; adaptive escalation remains useful when they do not.

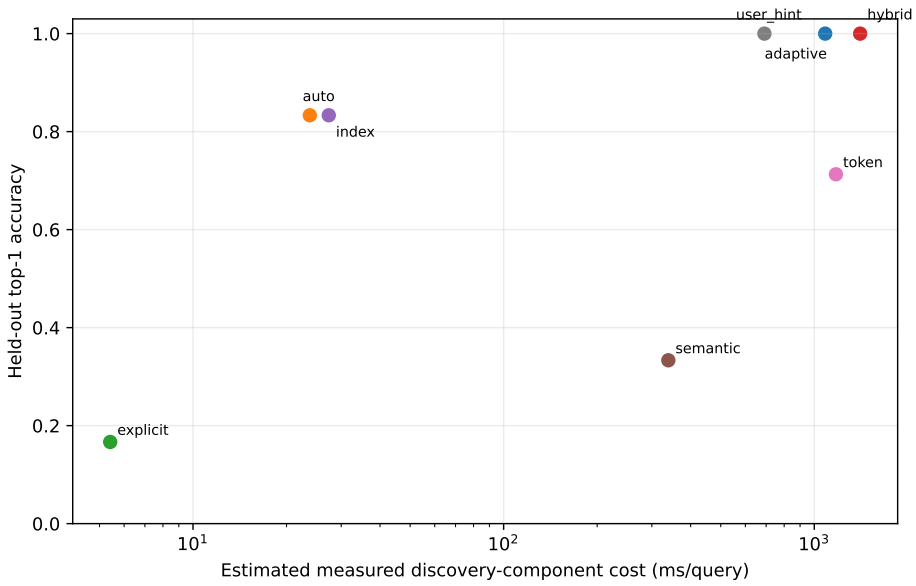

Figure 6: Largest-catalog ranking quality versus measured discovery-component cost. The log axis is needed because explicit and exhaustive hybrid paths differ by more than two orders of magnitude.

B.3 Logical catalog size is not active materialization

Every generated definition contains 18 accounting tokens. The largest catalog therefore contains 147,456 logical definition tokens, whereas resolving one resource requests 18 tokens, or 0.0122% of the catalog. Figure 7 makes that distinction visible. The number is an accounting upper bound on selected source text; M0 does not encode definitions into native K/V and therefore does not establish attention-memory or latency savings.

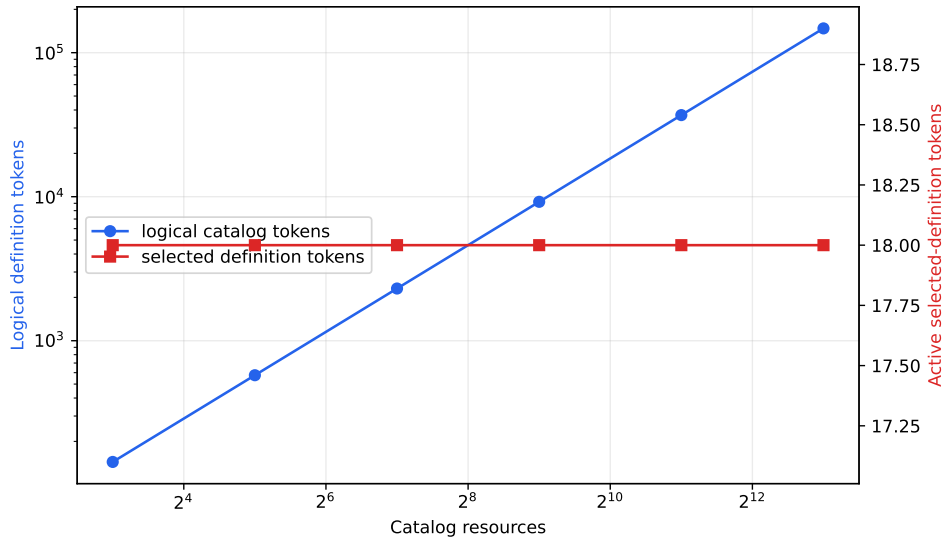

Figure 7: Logical catalog definitions grow with catalog size while the one-item selected-definition accounting remains at 18 tokens. The axes intentionally separate logical storage from active materialization.

B.4 Persistent indexing helps, but is not yet production-ready

At 8,192 resources, index construction takes 1.727 s, a full rebuild after a one-percent version mutation takes 1.753 s, and the estimated Python index payload is 5.27 MiB. Warm indexed lookup takes 27.4 ms and scores 966 candidates on average (11.8% of the catalog). By comparison, exhaustive semantic, token, and hybrid controls take 339, 1,174, and 1,405 ms. The index build amortizes to 44.6 ms/query after 100 uses and 29.1 ms after 1,000 uses.

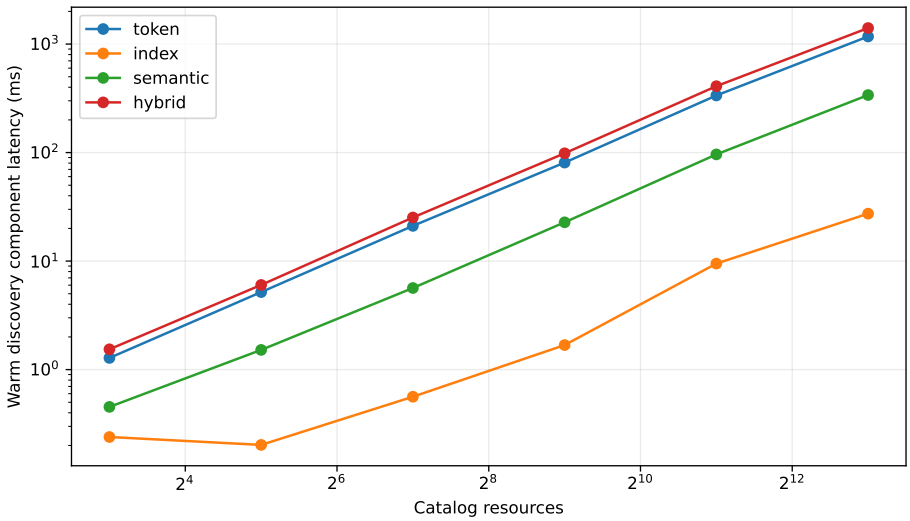

Figure 8: Warm CPU Python component latency. Exhaustive channels scale poorly; the indexed path narrows candidates but still touches about 11.8% at 8K.

These numbers establish an optimization target rather than a speed claim. Production token postings should avoid Python object traversal, semantic search should use an approximate vector index, unions should be deduplicated before scoring, and mutation should update affected postings rather than rebuild the entire immutable index. Cold/warm serving measurements must also include tokenization and downstream native-K/V work.

C What the M0 Gate Establishes

The mechanism satisfies four narrow requirements. First, typed resources can be resolved without exposing the full catalog to a language model. Second, different reference forms favor different channels. Third, request hints can match adaptive quality with fewer stages. Fourth, action-aware calibration can distinguish a correct ranking from a decision safe enough to use.

M0 alone does *not* satisfy the paper's broader context gate. Its 18-token active count is not a measured model working set. There is no model-side argument generation, observation-conditioned continuation, persistent-history recall, skill composition, or subagent inheritance. Its negative controls are small and its semantic control is idealized. We therefore treat M0 as resolver evidence rather than evidence that PRA improves agents.

D M1: A Causal Toy-Model Gate

D.1 Protocol separates discovery, generation, and execution

The synthetic language gives each tool an opaque identity and a compact definition. Semantics appear only in that definition. A held-out query states an action, object, family, and raw argument, but contains neither the resource identity nor one of four formatting schemas. The host first discovers a stable URI and binds it to a temporary slot such as `@0`. The model must emit `slot|schema|argument`; the host resolves the slot and accepts the call only when the selected URI, schema, and formatted argument are all exact. A deterministic observation is then supplied and the model must continue with the expected answer. No external side effect is performed.

This boundary matters. Discovery quality is measured by stable URI identity. The model is responsible for schema-conditioned call construction. The host is responsible for binding and validation. A correct retrieval cannot receive end-to-end credit for an incorrect call, and a memorized call string cannot receive selection credit for an incorrect resource.

D.2 Model, training, and causal conditions

The character-level decoder has 506,400 parameters, four layers, hidden width 96, feed-forward width 384, RoPE, and PRA native-K/V at layers 1 and 3. Each of five seeds trains for 3,000 steps with a mixture of native-memory examples (50%), directly presented selected definitions (30%), and eager eight-item catalogs (20%). Loss is applied only to the call and post-observation continuation. Evaluation uses eight examples per seed at catalog sizes 8, 32, and 128.

We compare oracle memory, idealized semantically discovered memory, shuffled memory, disabled memory, the selected definition directly in the native prompt, and the eager catalog. All conditions share model weights. Shuffling preserves memory presence and size but selects the wrong definition; disabling removes memory. Eager conditions that exceed the 512-token native window are recorded as context overflow and are not truncated into an easier task.

D.3 Selected native K/V is used causally

Table 7 reports the strict end-to-end metric: correct resource, exact call, and exact continuation. Discovered memory matches oracle memory at all three sizes. Shuffled and disabled conditions obtain zero, so performance cannot be explained by the query alone or by generic memory activation.

Table 7: M1 end-to-end success, averaged over five seeds and eight held-out examples per seed. A dash denotes native-window overflow rather than failure.

Catalog	Discovered	Oracle	Direct	Shuffled	Disabled	Eager
8	1.000	1.000	0.925	0.000	0.000	0.025
32	0.950	0.950	0.825	0.000	0.000	–
128	0.925	0.925	0.800	0.000	0.000	–

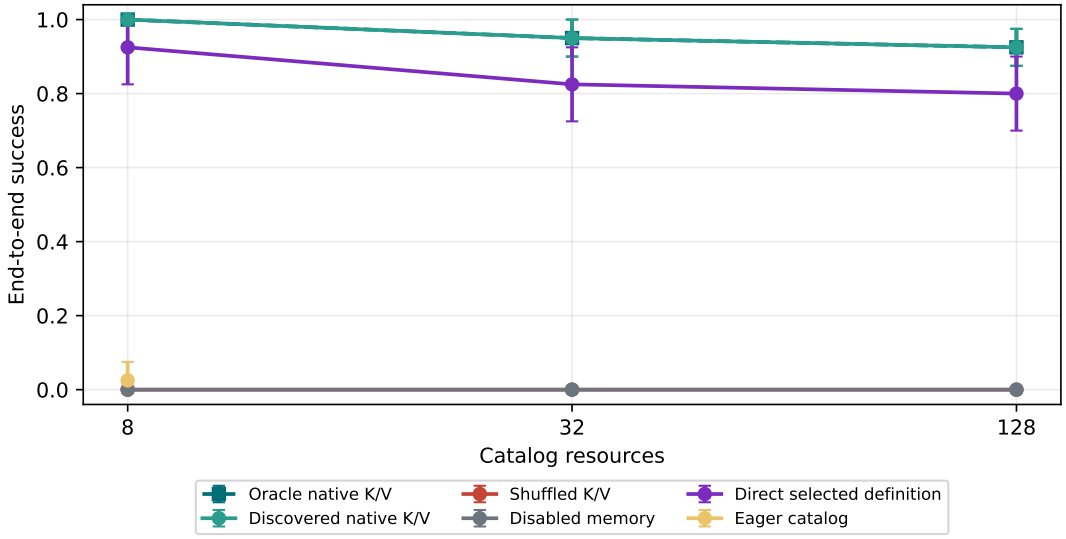

Figure 9: M1 end-to-end success. Oracle and discovered curves overlap; shuffled and disabled controls remain at zero. Eager evaluation is defined only where the complete catalog fits.

Every seed favors oracle memory over shuffled and disabled memory at every catalog size. With five paired seeds, however, the minimum attainable exact two-sided sign-flip value is $p = .0625$. Oracle and discovered memory have zero paired difference. Direct presentation is lower by .075 at size 8 and .125 at sizes 32 and 128, but those paired differences are not statistically decisive. We report the direction without claiming that PRA presentation is intrinsically better than direct presentation.

D.4 Logical growth is separated from active K/V

Figure 10 measures the neural working set that M0 could only account for. Logical catalog text grows with catalog size and crosses the native window before size 32. The selected native-K/V payload remains near one definition per PRA layer. Direct selected-definition prompts also re-

main bounded, making them the relevant non-PRA control. This is evidence of bounded active materialization in the toy implementation, not a production memory or latency claim.

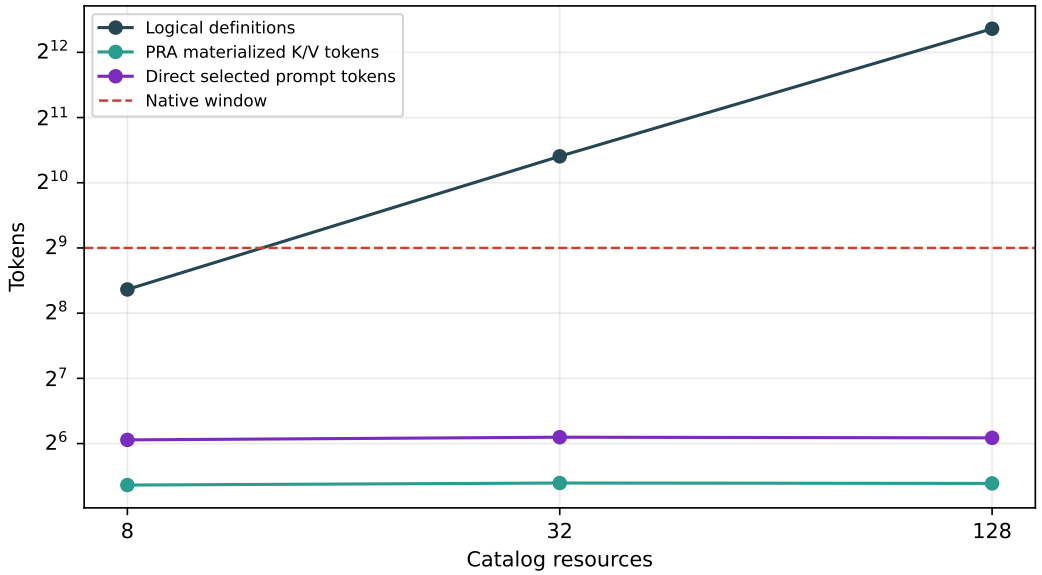

Figure 10: M1 logical catalog text, materialized native K/V per PRA layer, direct selected prompt length, and the 512-token native window. Logical storage and attention-visible working memory are distinct quantities.

The causal result remains narrow. The semantic discoverer is an idealized control, not a learned production index. The grammar is synthetic, the executor is pure and deterministic, and the decoder is trained specifically for this task. The experiment establishes that the selected native K/V carries necessary information through call construction and observation-conditioned continuation; it does not establish general tool use.

E M2–M4: From Selection to Typed Execution

E.1 Frozen pretrained bridge

M2 freezes Qwen3-0.6B at revision `c1899de289a04d12100db370d81485cdf75e47ca`. Four realistic single-tool tasks cross five deterministic prompt presentations. The model receives OpenAI-compatible function schemas and must emit one parseable `<tool_call>` with the exact function and arguments. Conditions expose the selected tool, the identical oracle tool, a same-family shuffled tool, an irrelevant tool, no tools, or all 18 catalog tools. The five presentations vary instruction wording and eager order; they are not independent model-training seeds.

Selected and oracle disclosure both obtain 1.000 strict success (20/20). Shuffled, irrelevant, and empty conditions obtain 0/20. Eager disclosure obtains 0.850 (17/20). Thus the selected schema causes the output and a small model can use it without task-specific training; the eager catalog is not an oracle and already introduces interference.

E.2 Safe execution and typed observations

M3 inserts a host boundary after generation. The parser treats model JSON as untrusted. The executor checks that the function was disclosed, the stable URI was authorized, required arguments and types match the schema, and write or destructive authority was granted separately. Only registered pure in-memory handlers can run. A result becomes

`!!ref:observation:<namespace>:<call-id>:v1!!`

with producer URI, result schema, call identity, tenant, and provenance. Tool outputs are therefore retrievable resources rather than irreversible transcript text. All 20 selected calls execute and create typed observations. In 17/20 continuations (0.850), the model includes every returned value. The remaining three are consumption failures, despite correct execution.

E.3 Reactive multi-tool composition

M4 uses three held-out workflows of length three, four, and five. Reactive JIT shows only the next discovered schema after each typed result; eager-required shows the complete evaluator-required set at every step; no-refresh leaves only the initial tool visible. The same model and prompt presentations are used. Strict task success requires every expected identity, exact arguments, accepted execution, and typed observation in order.

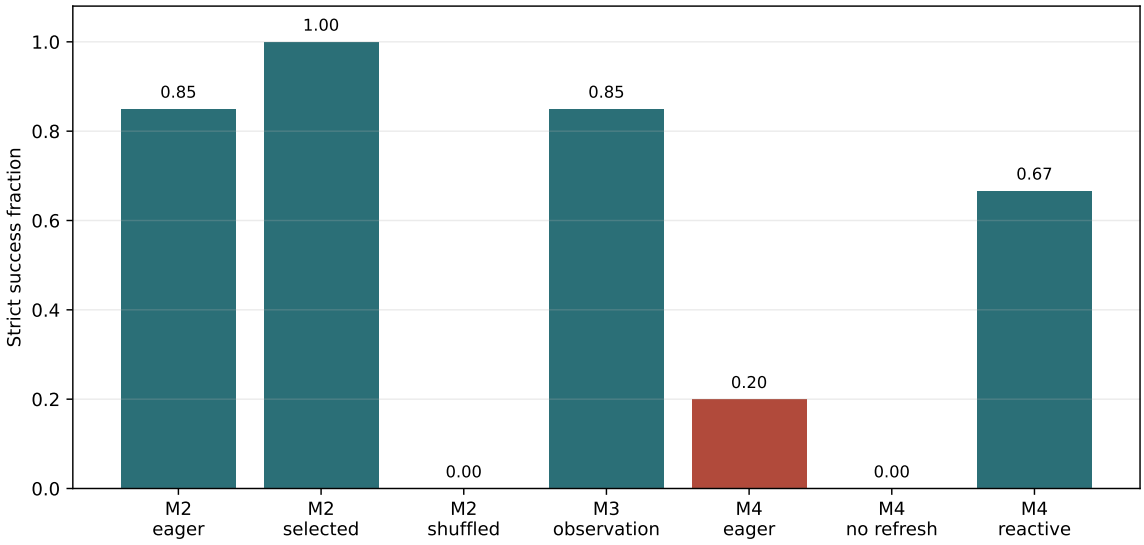

Figure 11: M2–M4 strict gates. Selected definitions cause exact pretrained calls; typed execution is reliable, while observation consumption remains imperfect. Reactive disclosure substantially outperforms both a static required set and a no-refresh control on the small workflow cohort.

Reactive JIT completes 0.667 (10/15), eager-required 0.200 (3/15), and no-refresh 0/15. The three- and four-step families account for the reactive successes; the five-step cross-domain repository workflow fails after its first result in every presentation. M4 is therefore a positive reactive-composition gate, not general long-horizon agent competence.

F M5: Speculative Capability Disclosure

M5 asks whether exposing a useful capability neighborhood before the first call improves planning relative to direct Top-1 or reactive retrieval. The 18-node graph has 171 typed edges. We compare all-tools eager (P0), direct Top-1 and Top-K (P1–P2), family/category, tag/keyword, and schema expansion (P3–P5), their bounded union (P6), reactive JIT (P7), horizon-matched speculative disclosure (P8), and the exact required set (P9). Labels distinguish required, useful, related-but-unnecessary, irrelevant, and unsafe tools.

Table 8: M5 capability-set and execution results over three workflows. Recall and initial tool counts are averaged across tasks. Model success has 15 trials for P1, P6–P9; a dash denotes a set-only policy.

Policy	Required recall	Initial tools	Unsafe	Model success
P0 all eager	1.000	18.00	1.00	–
P1 direct Top-1	.261	1.00	0	.000
P2 direct Top-K	.822	4.00	0	–
P5 schema graph	.628	3.67	0	–
P6 combined graph	0.933	6.67	0	0.000
P7 reactive JIT	.261	1.00	0	.667
P8 speculative matched	.628	3.67	0	.000
P9 oracle capabilities	1.000	4.00	0	.200

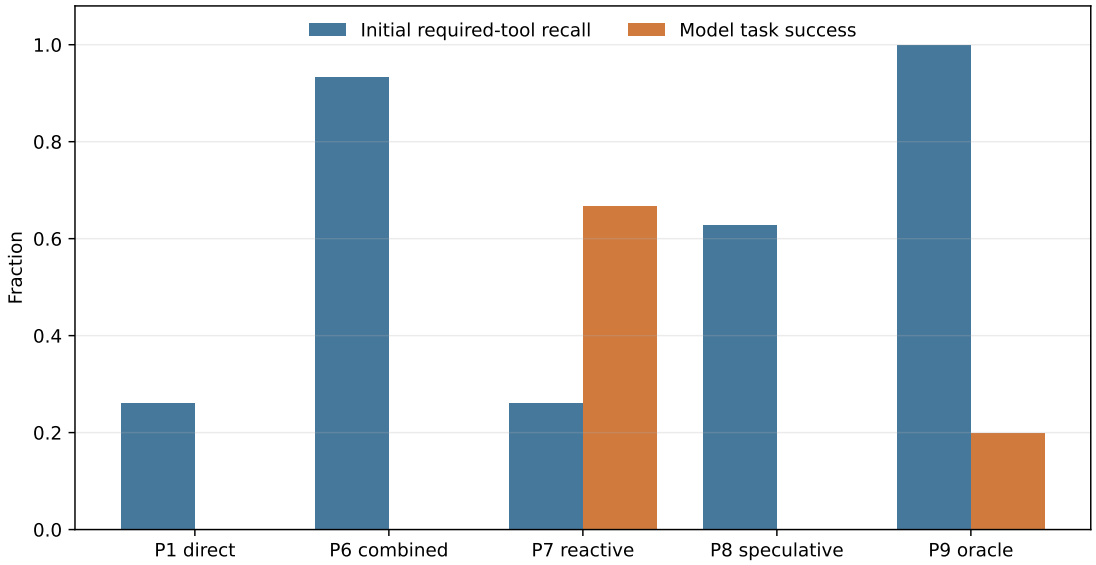

Figure 12: Initial required-tool recall is not task success. The combined graph recovers most required capabilities without unsafe exposure, yet the frozen model does not execute successfully from the larger static set.

The M5 stop/go gate is negative. P6 fully covers two workflows and reaches .933 mean required recall, but static P6 and P8 disclosure obtain 0/15 model success. Even P9 obtains only 3/15, whereas P7 reactive JIT obtains 10/15. The experiment does not support training an adaptive

disclosure controller. The fixed graph remains useful for auditable candidate construction, but a useful capability set expands the action space; it does not solve action sequencing.

G M6: Optional PRA-Native Resource Discovery

Native Q/K is not treated as another ordinary SDK function. In a generic deployment, the agent and resolver sit outside an opaque model API and cannot read hidden states, native query vectors, key vectors, or PRA cache entries. M6 therefore defines four explicit arrangements: co-located execution; a same-host shared-memory index that exports only $z_q = W_q q$; a model-server resolver endpoint that returns typed URIs and scores; and replicated query computation requiring the same model, tokenizer, layer, projection, and position treatment. Only the co-located mechanism is executed here. The SDK also implements the projected-query payload and identity-only server reply; raw Q/K is not persisted in artifacts.

At frozen Qwen layer 27, 1,625 tokens from 18 structured definitions are encoded with explicit name, description, parameter, return, side-effect, and tag boundaries. Five prompt presentations cover four single-tool and three multi-tool tasks. Baselines include token and indexed lexical routing, a signed-hash control, input-embedding means, native mean K, full token QK, and a lexical/native fusion. The rank-16 and rank-8/eight-centroid projections are the five Paper-2.8 HotpotQA/QASPER checkpoints applied zero-shot; they receive no tool supervision.

Table 9: M6 co-located discovery. Multi-step recall uses a budget equal to the workflow horizon. Bytes are declared persistent routing-index payloads, not Python/process overhead or backing native K/V.

Mode	Single Top-1	Multi recall	Successor recall	Index bytes
Token	.800	.889	1.000	13,427
Index	1.000	.862	1.000	13,427
Signed hash	.800	.670	.906	9,216
Input-embedding mean	.650	.619	.572	36,864
Lexical + rank-16	.800	.690	.689	65,427
Native mean K	0.000	.439	.611	36,864
Native token QK	.000	.439	.611	3,328,000
Paper-2.8 rank-16	.000	0.222	.167	52,000
Paper-2.8 rank-8/8-centroid	.000	.372	.167	2,304

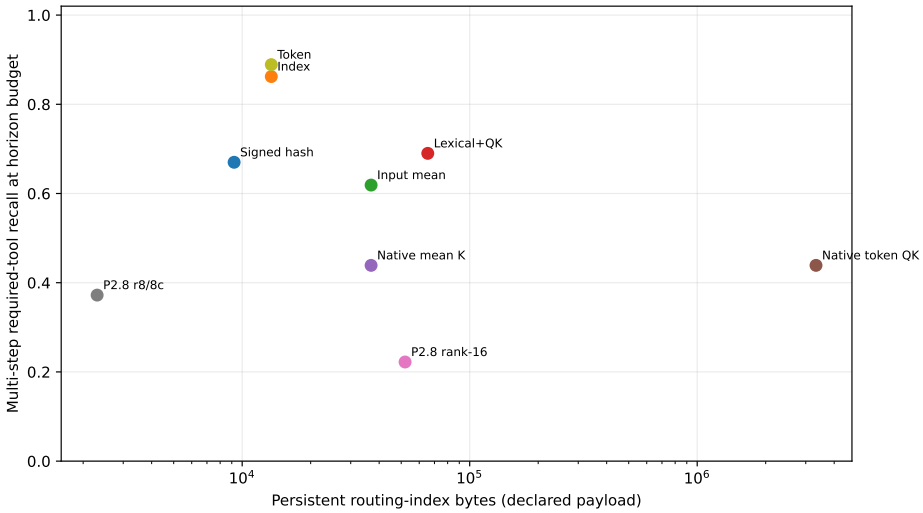

Figure 13: M6 multi-step retrieval quality against routing-index payload. The compact transferred projection is inexpensive, but lexical channels dominate quality on this catalog. Full native-token scoring is both worse and larger.

M6 also stops negatively. Native query/key state does not improve this tool catalog, and the QA-trained low-rank map does not transfer to capability language. The lexical/native fusion is below either lexical channel alone. Consequently the supported generic architecture remains external explicit/token/index discovery. Model-server native routing is an optional research interface whose tighter coupling is not justified by these results.

H M6.5: External Semantic-Hard Discovery

H.1 Frozen benchmark and split discipline

M6 answers a canonical-language question; it does not show that lexical search is sufficient for sloppy intent. We therefore freeze 306 single-tool requests over the same 18-tool catalog. Each identity has one held-out test and one validation realization in H0 (canonical), H1 (synonym), H2 (colloquial), H3 (implicit goal), H4 (bounded context plus shorthand), and H5 (Portuguese, Spanish, or French against English tool definitions). Eighteen additional audit rows cover the catalog. Validation and test contain 144 identities each; the complete query fingerprint is `e348334ad2362f284a217f1f852f175917a17540dd403f79953d7d2a7ebd7adb`. For H2 and later, tests reject exact tool names and aliases. H4 gives every channel the same context. No test wording selects an encoder, representation, fusion weight, or confidence threshold.

This is a controlled, project-authored fixture rather than a naturally sampled user corpus. Its multilingual mappings and tool tags share the benchmark's typed concept inventory. Their accuracy measures the value of registration-time semantic metadata under those declared assumptions, not open-domain translation or multilingual generalization. The benchmark evaluates discovery only; the multi-step planning and execution boundaries remain those measured in M4–M5.

H.2 External policies

The concept map resolves domain-bounded operation/object phrases and retains language, source, and weight for every expansion. Tool registration emits four representations: description; name plus description; a deterministic card with purpose, operation, object, input/output types, and side effect; and three separate vectors for those fields. Three compact encoders are selected on validation only: all-MiniLM-L6-v2, BGE-small-en-v1.5, and multilingual MiniLM-L12-v2, all pinned by revision. Validation selects BGE with description cards and context-plus-query for English, and multilingual MiniLM with structured cards and query-only encoding for H5. Tool vectors are persisted; third-party weights are not copied into the repository.

The policy ladder compares token overlap (P0), BM25 (P1), typed dictionary expansion (P2), registration-time tags (P3), their lexical combinations, English and multilingual embeddings, a validation-frozen hybrid, and an identity oracle. A staged P10 resolver tries lexical, dictionary/tags, and the embedding hybrid before asking. Its four thresholds are fitted on validation.

Table 10: M6.5 results on 144 frozen test requests. Latency is the median measured end-to-end resolver time for this 18-candidate Python implementation; embedding registration is amortized.

Policy	Top-1	MRR	Recall@3	Median ms
Token	.257	.398	.403	8.6
BM25	.347	.485	.514	8.6
Typed dictionary	.729	.812	.868	8.6
Registration tags	.743	.823	.875	8.6
English embedding	.563	.682	.750	12.3
Multilingual embedding	.556	.690	.785	10.8
Dictionary/embedding hybrid	.729	.809	.875	12.3
Identity oracle	1.000	1.000	1.000	8.6

Tags improve Top-1 over BM25 by .396 (paired identity bootstrap 95% interval [.299, .500]; exact discordance $p < 10^{-10}$). The gain is not uniform. English BGE is strongest on H3 implicit goals (.722), tags on H4 context (.556), and curated tags are perfect on H5 because all three languages are represented in the frozen concept inventory. The hybrid improves H2 to .667 and H5 to .963 but does not dominate each specialized channel. This is why the deployment policy is a calibrated ladder, not a claim that one embedding replaces lexical metadata.

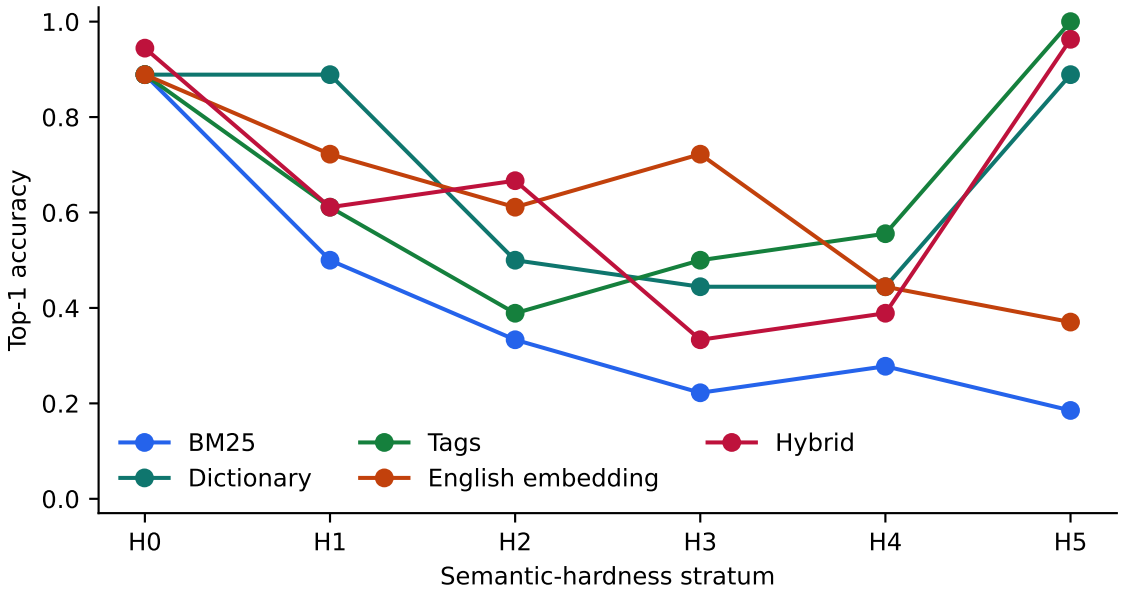

Figure 14: External discovery by hardness. Lexical quality declines as canonical wording disappears; typed metadata and embeddings recover different strata.

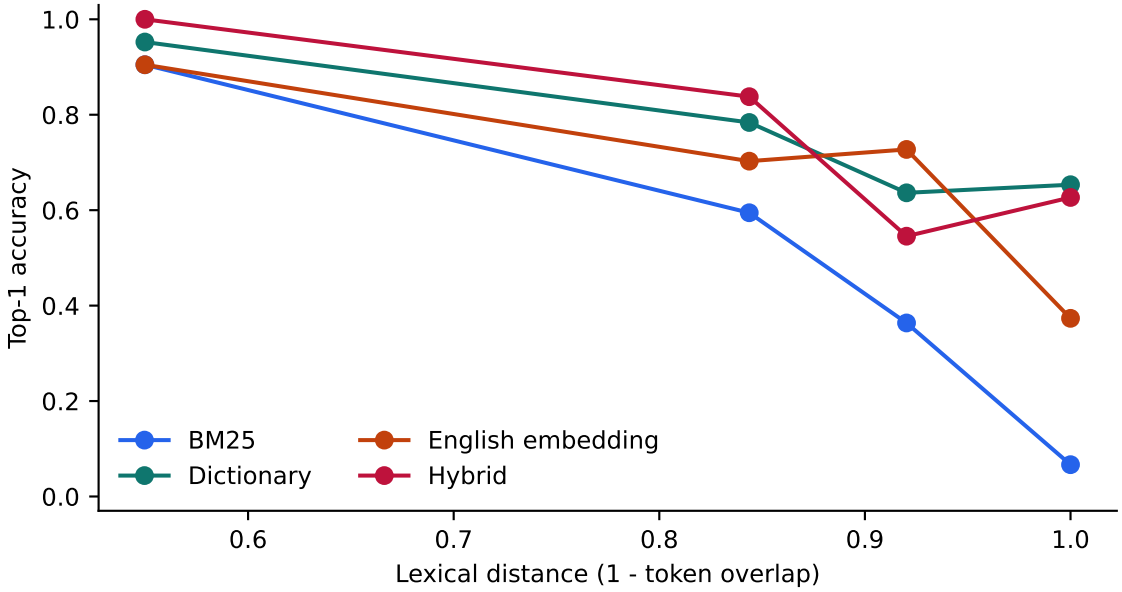

Figure 15: Required quality-versus-distance analysis. Bins contain 21, 37, 11, and 75 test identities from lower to higher lexical distance. Dictionary and hybrid channels retain substantial accuracy when token overlap is zero.

H.3 Selective action and deployment cost

On test, staged P10 selects 86 of 144 requests and asks on 58: coverage is .597, selective accuracy is .907, actionable accuracy is .542, and the false-action rate is .056. No selected top tool is in

the row's unsafe set. Its Brier score is .135 and 10-bin ECE is .098. These held-out numbers are weaker than validation (.949 selective accuracy), which is a reason to retain ASK rather than lower thresholds after seeing test outcomes.

The 384-dimensional persistent tool vectors occupy 27.6 kB. The selected BGE encoder has 133.4 MB of parameters, loads into the process in 3.75 s, and encodes the full 306-query cohort in 1.13 s on the recorded CUDA host; its median per-query resolver path is 12.3 ms versus 8.6 ms for dictionary/tags. The multilingual encoder is larger (470.6 MB) and is not selected for the generic fusion.

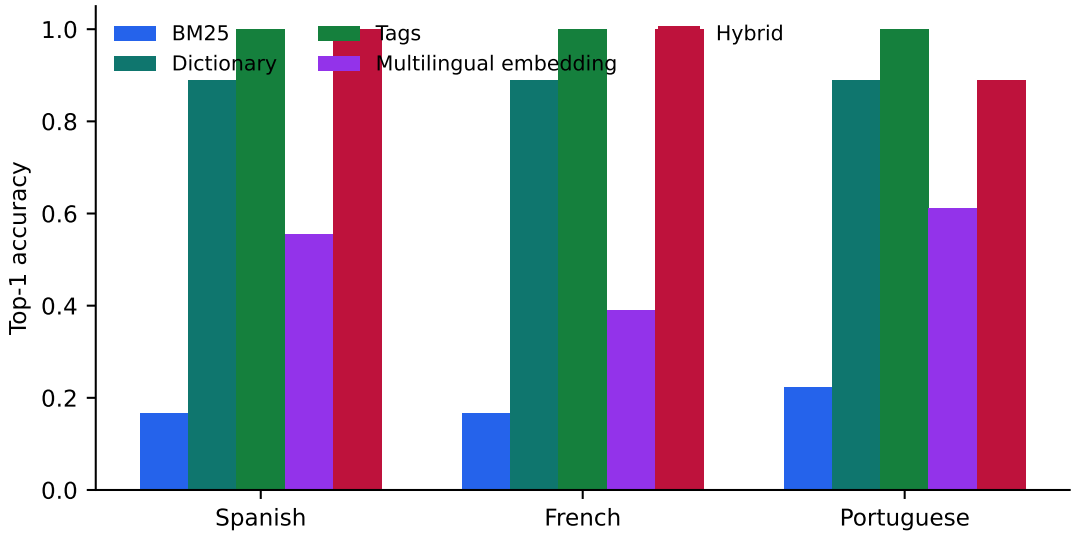

Figure 16: H5 retrieval against English tool definitions. The curated dictionary and tag channels encode the three tested languages by construction; the multilingual encoder is an independent model-backed control.

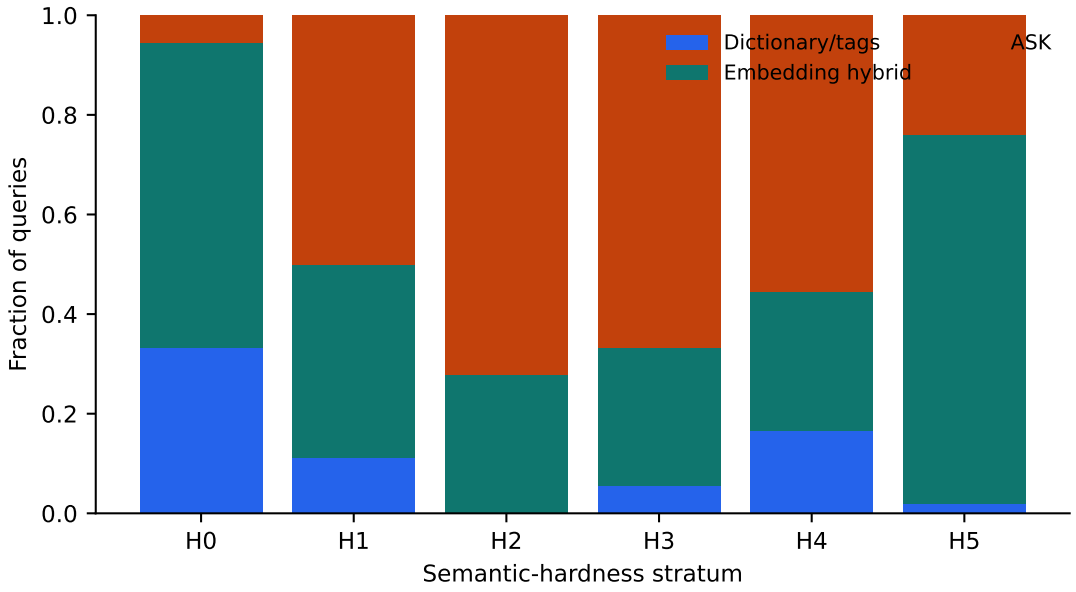

Figure 17: Validation-calibrated P10 stage outcomes on test. More semantic-hard rows reach the hybrid or ASK stages instead of forcing a low-confidence call.

I M7: Native Geometry on the Frozen Hard Set

M7 reuses all 144 test identities without changing wording, tool cards, or external policies. Frozen Qwen3-0.6B layer-27 pre-RoPE state supplies native mean K and full token QK. The Paper-2.8 rank-16 ensemble and rank-8/eight-centroid ensemble are transferred from five HotpotQA/QASPER checkpoints without tool supervision. Query encoding is charged online; raw native state is not persisted.

Table 11: M7 matched semantic-hard comparison. Model bytes include the frozen Qwen required to reproduce native queries; index bytes exclude backing model weights. The rank-8 timing includes the current per-query centroid construction and is not a production index latency.

Mode	Top-1	MRR	Recall@3	Median ms
BM25	.347	.485	.514	8.6
Tags	.743	.823	.875	8.6
English embedding	.563	.682	.750	12.3
External hybrid	.729	.809	.875	12.3
Native mean K	.056	.233	.313	116.7
Native token QK	.062	.220	.181	115.9
Paper-2.8 rank-16	.056	.194	.167	122.9
Paper-2.8 rank-8/8-centroid	.049	.190	.167	1,658.4

The native Top-1 range, .049–.063, contains the uniform catalog chance rate $1/18 = .056$. Tags exceed native mean K by .688 (paired bootstrap 95% interval [.604,.764]) and full token QK by .681 ([.597,.764]). Thus the harder wording does not reveal a frozen native advantage hidden by M6's canonical queries. This result is narrower than "native Q/K cannot encode tool intent": it says that one frozen layer, two direct reducers, and QA-trained low-rank projections do not expose

it zero-shot for this catalog.

Tool-specific native training does not open. External accuracy leaves oracle headroom, but none of the four frozen native diagnostics supplies the required positive signal, and reproducing native queries requires a 1.19 GB model plus co-location or a model-server extension. The next justified experiment would need tool-supervised evidence independent of this test set, not adaptation to the 144 held-out requests.

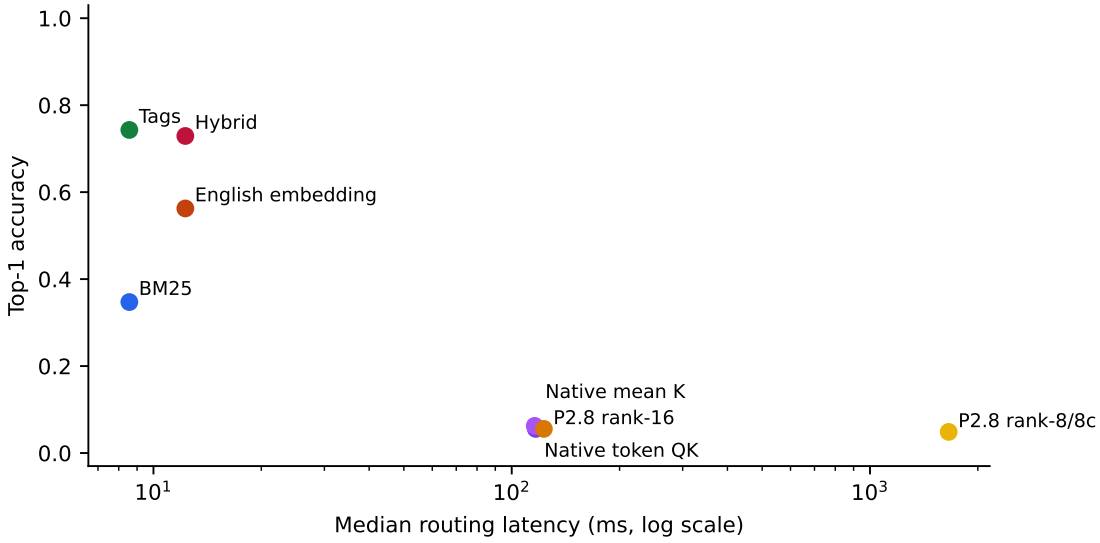

Figure 18: M7 quality–latency frontier on matched queries. External typed semantics dominates the tested frozen native modes in both quality and online latency on this small catalog.

J M8: Automatic Tool Records and Record-Aware Materialization

J.1 Ordinary callables become typed resources

M6.5's curated registration metadata raises an adoption question: must every function author learn a PRA-specific vocabulary? M8 adds `tool_record_from_callable`, which inspects but never invokes a Python callable. It preserves module and qualified name, sync/async status, signature, parameter kinds and defaults, input and return annotations, and descriptions parsed from Google, NumPy, or Sphinx-style docstrings. A provider-independent `ToolSchema` exports to provider formats only at the boundary. Its type cache covers primitives, generic containers, optional and union types, `Literal`, enumerations, dataclasses, `TypedDict`, annotated classes, and Pydantic when installed; no optional dependency is required.

Semantic enrichment remains auditable. Function-name terms, parameter and return types, docstring terms, canonical operation/object concepts, and dictionary expansions retain source and deterministic weight. Curated and automatic tags occupy separate fields. Generic verbs such as `get` and `read` can establish operation type without receiving the same lexical weight as `delete` or `update`. Name-inferred destructive operations are conservatively hidden from ordinary disclosure, but this hint does not grant or replace host execution authority.

E1 aligns 18 ordinary annotated callables with the existing rich catalog and uses the same 144 frozen test requests. Fusion weights are selected only on the 144 validation requests. The

automatic hybrid obtains 0.826 Top-1 and 0.951 Recall@3, compared with 0.806 Top-1 for the rich manual hybrid. Thus automatic metadata retains all measured gain over its lexical baseline in this controlled catalog. This is evidence for a low-annotation registration path, not evidence that docstrings are always complete or that inferred risk metadata is authorization.

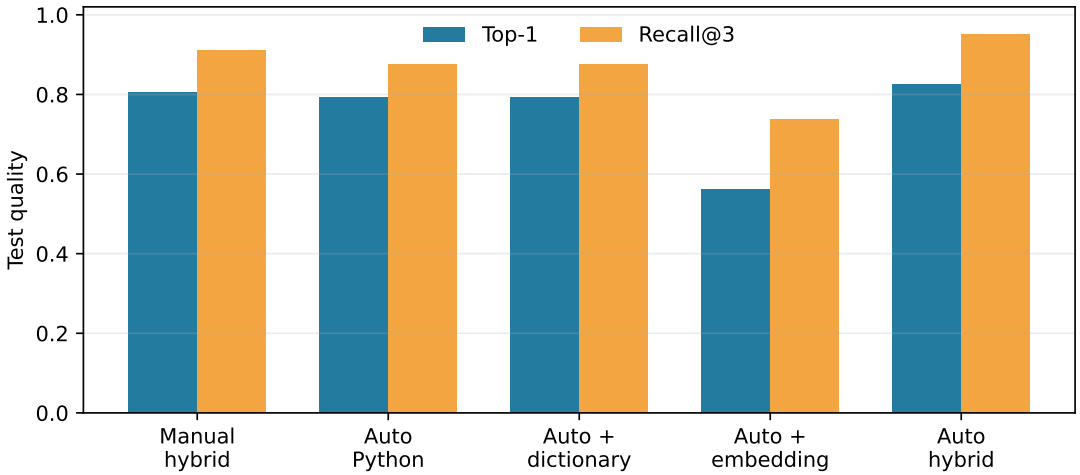

Figure 19: E1 frozen-test quality. Ordinary signatures and docstrings recover the rich catalog result after validation-only fusion selection.

J.2 A zero-configuration ladder identifies the useful signals

The combined E1 result does not show which automatically derived fields matter. We therefore freeze one `ToolRecord` per callable and score every policy against the same 144 test requests. The record contains only function and qualified names, module, signature, parameter and return annotations, descriptions parsed from the docstring, and deterministic fields derived from them. Every derived term records whether it came from a name, docstring, parameter, return, type schema, dictionary expansion, inferred operation or object, automatic tag, or embedding field. No condition receives a per-tool alias, tag, operation, object, or capability edge.

Table 12 gives the complete ladder. Raw BM25 reaches 0.403 Top-1. Reweighting automatically extracted keywords alone raises Recall@3 but lowers Top-1 to 0.306; extraction is therefore not itself a semantic resolver. The global, tool-independent synonym dictionary supplies the largest isolated gain, reaching 0.750 Top-1. Adding inferred concepts and tags by elementwise score maxima does not improve that ranker in isolation. The selected name-plus-description embedding is complementary rather than sufficient. A validation-selected mixture of synonym, automatic-tag, and embedding scores reaches 0.812 Top-1 and 0.938 Recall@3, versus 0.806 and 0.910 for manually enriched registration. The one-row Top-1 difference is not evidence that automatic metadata is generally superior; it is a parity result on one controlled catalog.

Policy	Manual	Syn.	Concepts	Embed.	Top-1	R@3	R@5
A0 Raw BM25	no	no	no	no	0.403	0.556	0.639
A1 Auto keywords	no	no	no	no	0.306	0.597	0.681
A2 Keywords + synonyms	no	yes	no	no	0.750	0.882	0.910
A3 Inferred concepts	no	yes	yes	no	0.632	0.882	0.903
A4 Automatic tags	no	yes	yes	no	0.618	0.875	0.903
A5 Compact embedding	no	no	no	yes	0.556	0.764	0.840
A6 Automatic hybrid	no	yes	yes	yes	0.812	0.938	0.958
A7 raw union ($K = 10$)	no	yes	yes	yes	–	0.882	0.910
Manual-rich ceiling	yes	yes	yes	yes	0.806	0.910	0.958

Table 12: Frozen-test automatic discovery ladder. A7 has no unique Top-1: R@3 and R@5 evaluate caps three and five, while validation selects raw union at $K = 10$ for its maximum set recall. "Manual" denotes per-tool authored semantic metadata; all A0–A7 fields come from the same callable record.

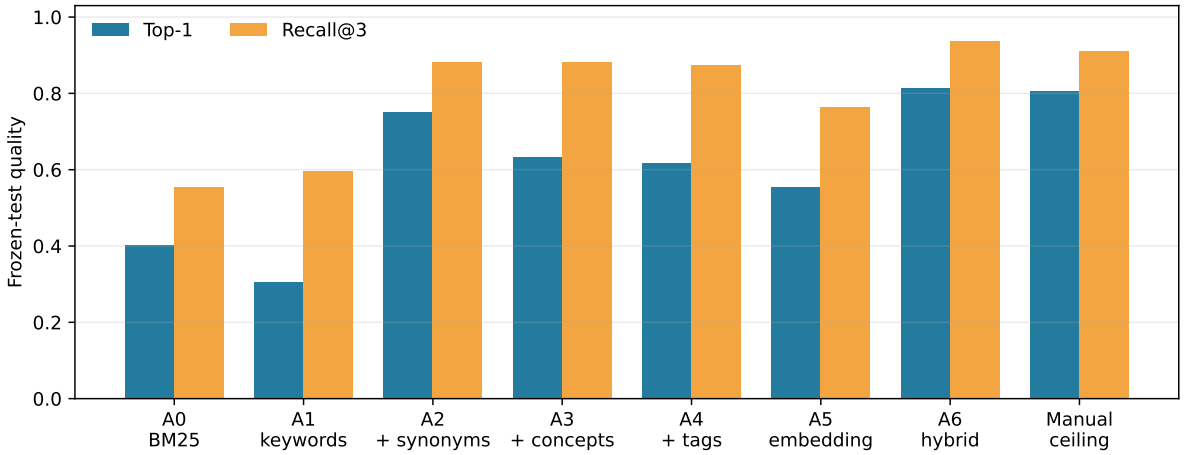

Figure 20: A0–A6 contribution ladder and the manual-rich control. A5 is an independent embedding condition rather than an incremental addition to A4; A6 is the validation-frozen fusion.

The hardness decomposition in Figure 21 clarifies the complementarity. BM25 is already strong on direct H0 requests but degrades on implicit and contextual requests. The generic dictionary dominates the multilingual H5 fixtures, whereas the English compact embedding is strongest on H2–H4 paraphrastic and implicit English requests. Their fusion is therefore more stable than either channel alone. These multilingual rows test the frozen dictionary coverage, not open-domain translation.

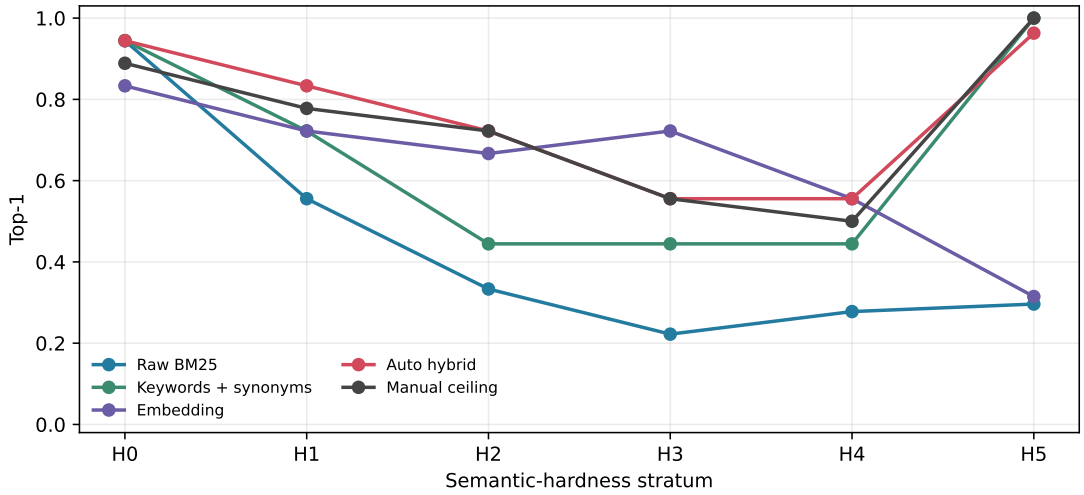

Figure 21: Top-1 by frozen semantic-hardness stratum. The automatic hybrid tracks the manual ceiling while using different channels for lexical, implicit, and multilingual requests.

One-at-a-time source controls locate practical SDK requirements. Removing docstring terms lowers A1 Recall@3 from .597 to 0.521; type/schema terms add 0.042 Recall@3, although they do not improve A1 Top-1 and do not alter A4. With no docstring, embedding Top-1 falls from .556 to 0.375. Replacing descriptive names with opaque identifiers while retaining the docstring and signature yields 0.507. Thus registration remains useful with weak names, but documentation quality matters, especially to the embedding fallback. Figure 22 reports these controlled variants; they do not modify the frozen request text.

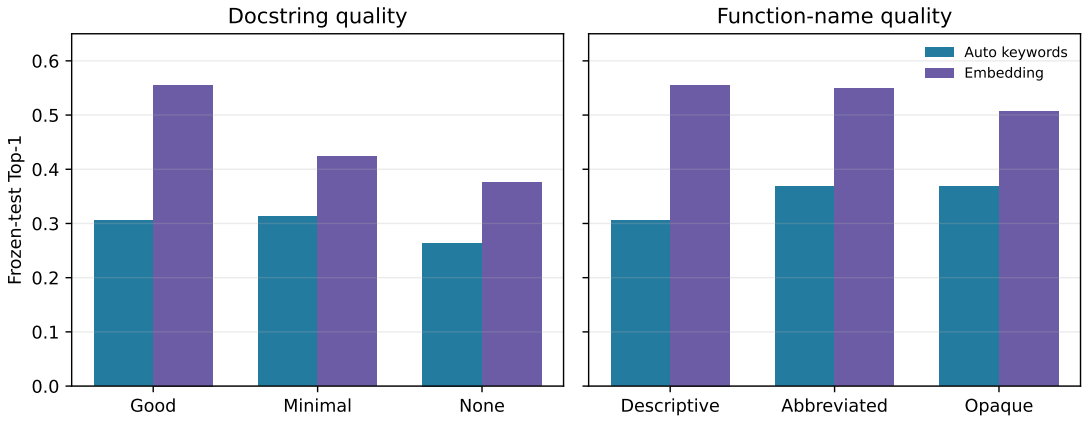

Figure 22: Sensitivity to callable documentation and naming. Each intervention changes only the named source while preserving signatures, target identities, and requests.

J.3 Candidate palettes do not reduce to one winner

The resolver can now return a bounded `CandidateSet` rather than only a Top-1 URI. Every candidate retains all channel hits and its admission source; deduplication uses stable URI. E2 sweeps budgets $K \in \{1, 2, 3, 4, 5, 6, 8, 10\}$ and compares the best single channel, normalized score fusion, raw union, and a diversity-aware union that admits one unseen candidate from each enabled chan-

nel before filling remaining slots. Metrics include required/useful recall, useful precision, unsafe exposure, schema tokens, and provenance diversity.

At $K = 4$, fused Top-K reaches 0.882 required-tool recall versus 0.847 for diversity union. Recall rises with larger palettes, but useful precision falls and labeled unsafe exposure increases. The requested default-union gate is therefore negative: candidate sets are first-class and union remains configurable, while validation-selected fusion remains the supported semantic palette policy for this cohort.

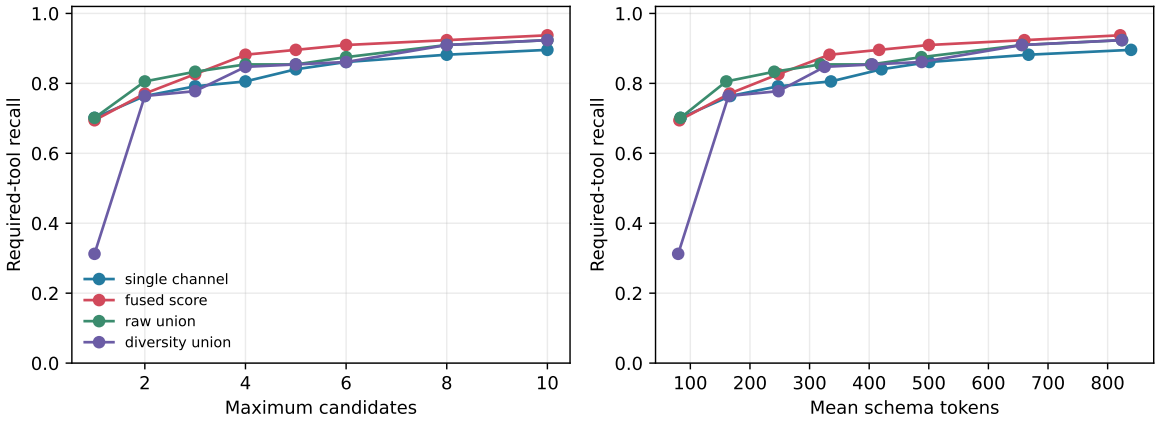

Figure 23: E2 required-tool recall against candidate count and serialized schema cost. Diversity preserves channel representation but does not dominate fused Top-K at matched budgets.

The isolated A7 rerun reaches the same decision with five strictly automatic channels: BM25, extracted keywords, keyword-plus-synonym scores, automatic-tag/concept scores, and the selected embedding. At $K = 4$, fused Top-K obtains 0.938 required-tool recall, versus 0.910 for raw union and 0.861 for diversity union. Validation selects raw union at $K = 10$ if recall is optimized alone; on test it reaches 0.993 recall but exposes at least one labeled unsafe candidate on 0.868 of requests and serializes 811 mean schema tokens. Channel complementarity is real but sparse: 8 of 144 targets are recovered only by the embedding, 3 only by an automatic lexical/concept channel, 129 by multiple channels, and 4 by none at per-channel $K = 3$. Larger set size, rather than broad unique recovery, explains most of the raw-union frontier.

The preregistered automatic-union JIT gate therefore remains closed. No new generative run is performed: on validation, neither union variant materially exceeds matched-budget fusion while holding unsafe exposure within one percentage point. This is an intentional stopped experiment, not missing data. Candidate provenance and union remain exposed by the SDK for diagnostics and future cohorts, but validation-frozen fusion remains the default.

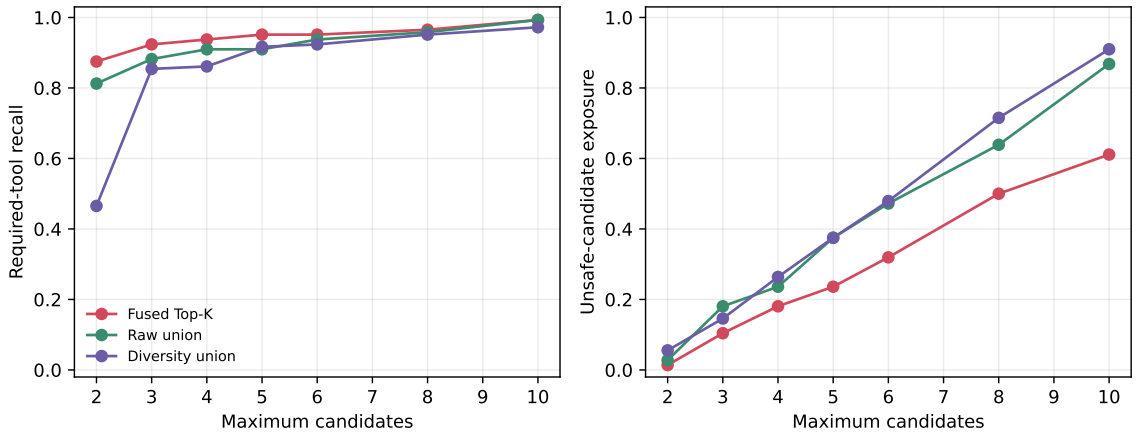

Figure 24: A7 frozen-test frontier. Union does not dominate fused Top-K in required-tool recall and raises unsafe-candidate exposure rapidly as the palette grows. Discovery exposure is diagnostic; host authorization remains a separate execution boundary.

J.4 Semantic confusion changes the large-catalog result

M0 established that a typed index can locate opaque resources in catalogs up to 8,192 entries. That result did not test whether zero-configuration semantic channels preserve ranking quality when thousands of plausible callable descriptions compete for the same intent. The scale follow-up therefore keeps the 18 target callables and 144 frozen H0–H5 test requests, then constructs catalogs of 32, 128, 512, 2,048, and 8,192 ordinary Python callables under five catalog seeds. Each added callable shares exactly one canonical facet with a target—its operation or its object—but never both. Names, docstrings, signatures, annotations, and modules are inspectable by the same registration path as the targets. Every fifth eligible distractor uses a destructive operation on the anchor object so unsafe exposure remains measurable. No generated distractor receives a manual tag or target label at ranking time.

The evaluation reruns A2 lexical-plus-synonym scoring, A5 compact embeddings, A6 with the representation and weights frozen on the original 18-tool validation set, and A7 equal fusion, raw union, and diversity union. Candidate budgets $K \in \{2, 3, 4, 5, 6, 8, 10, 20\}$ are matched exactly. Table 13 reports seed means at $K = 10$; "Union P" is useful precision and "Unsafe" is the fraction of requests whose candidate palette contains at least one labeled destructive non-target. Exposure is diagnostic and never grants execution authority.

Catalog	A2 Top-1	A5 Top-1	A6 Top-1	A6 R@10	Union R	Union P	Unsafe	Tokens
32	0.729	0.529	0.787	0.967	0.944	0.153	0.414	916
128	0.629	0.394	0.675	0.914	0.879	0.120	0.399	1026
512	0.564	0.272	0.546	0.850	0.801	0.101	0.208	1050
2,048	0.472	0.229	0.421	0.692	0.715	0.085	0.125	1057
8,192	0.451	0.206	0.338	0.553	0.662	0.078	0.110	1059

Table 13: Five-seed semantically confusable callable-catalog scaling. A2–A6 columns report ranking quality. Union columns report raw A7 union at matched $K = 10$; context tokens serialize complete selected schemas.

The original frozen hybrid remains strongest at 32 and 128 tools. Its Top-1 advantage disappears

by 512 and reverses at 2,048: at 8,192, A2 lexical evidence reaches 0.451 Top-1 while A6 reaches 0.338. The compact embedding alone falls to .206. Semantic similarity accumulates plausible neighbors as the catalog grows, whereas exact lexical evidence retains a sharper identity signal. Figure 25 shows that this is a gradual ranking failure, not a threshold artifact.

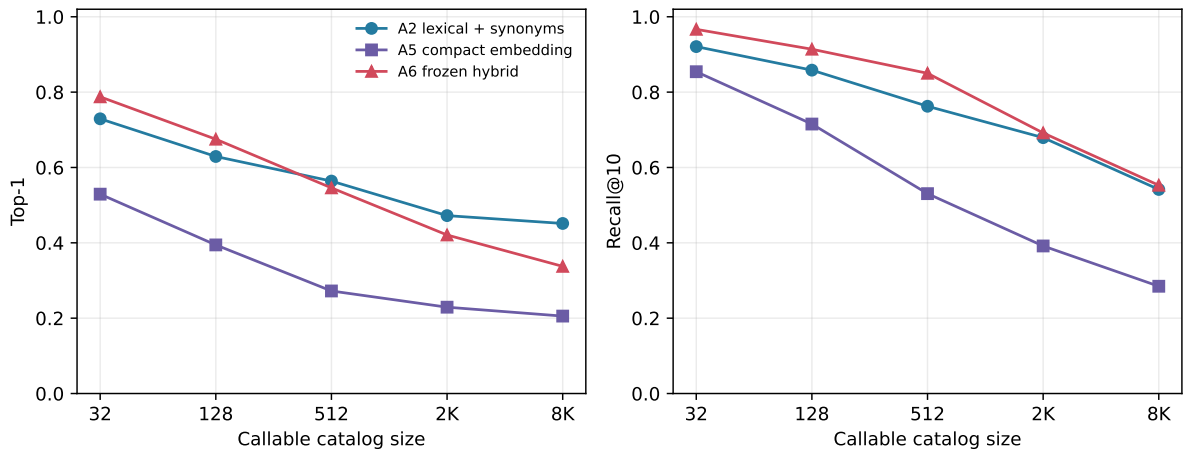

Figure 25: Top-1 and Recall@10 under five seeded callable catalogs. The A6 mixture is frozen rather than retuned at each scale.

Channel diversity nevertheless becomes useful for bounded recovery. At 8,192 tools and $K = 10$, raw union reaches 0.662 required-tool recall, compared with 0.553 for A6. The gain is not free: useful precision is 0.078, the palette exposes a labeled unsafe candidate on 0.110 of requests, and full schemas consume 1059 tokens on average. The declining unsafe-exposure rate with catalog size does not imply safer catalogs; under fixed K, increasingly many non-destructive confusable candidates displace destructive ones from the visible palette. Figure 26 therefore reports recall and exposure together.

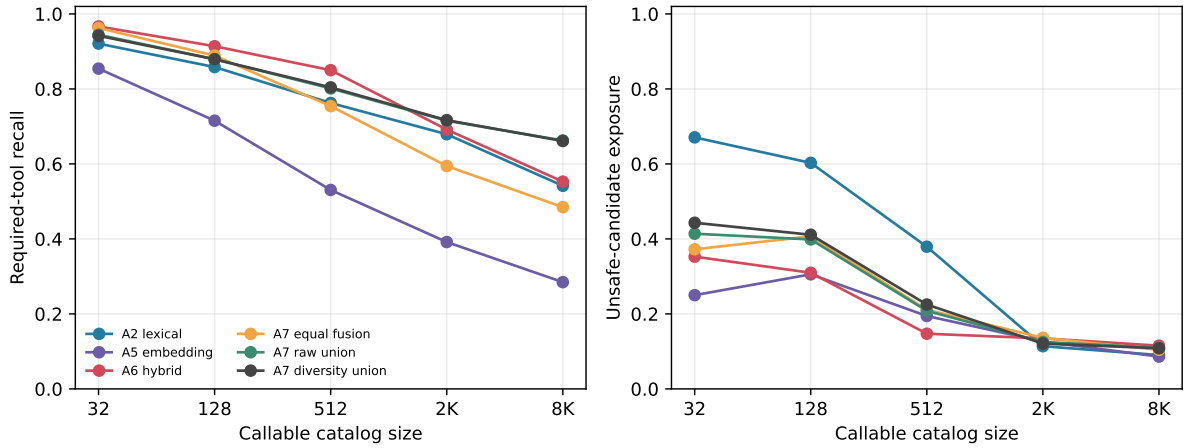

Figure 26: Matched-$K = 10$ required-tool recall and unsafe-candidate exposure. Raw and diversity union preserve more large-catalog recall, but neither is a safe or precise default disclosure policy.

Figure 27 exposes the 8K budget tradeoff rather than selecting $K = 10$ after the fact. Union's recall advantage grows with palette size while useful precision falls; $K = 20$ is a diagnostic upper

bound, not a recommended disclosure budget.

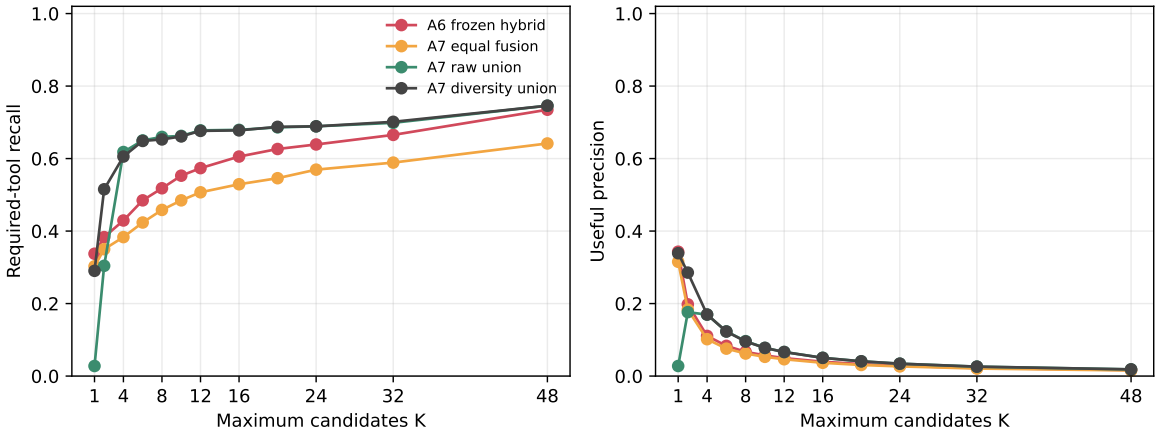

Figure 27: The 8,192-tool matched-budget frontier. Candidate-set recall and useful precision must be read together.

The Python reference implementation scales approximately linearly. At 8,192 tools, exhaustive BM25 scoring takes 138.7 ms/query and tensorized A7 candidate construction takes 30.9 ms/query after channel scores are available. Discovery components occupy 50.7 MiB, while serializing the entire logical catalog would require 895909 tokens. These measurements establish the cost shape, not a production latency claim: an inverted lexical index and approximate semantic index should narrow candidates before exact reranking.

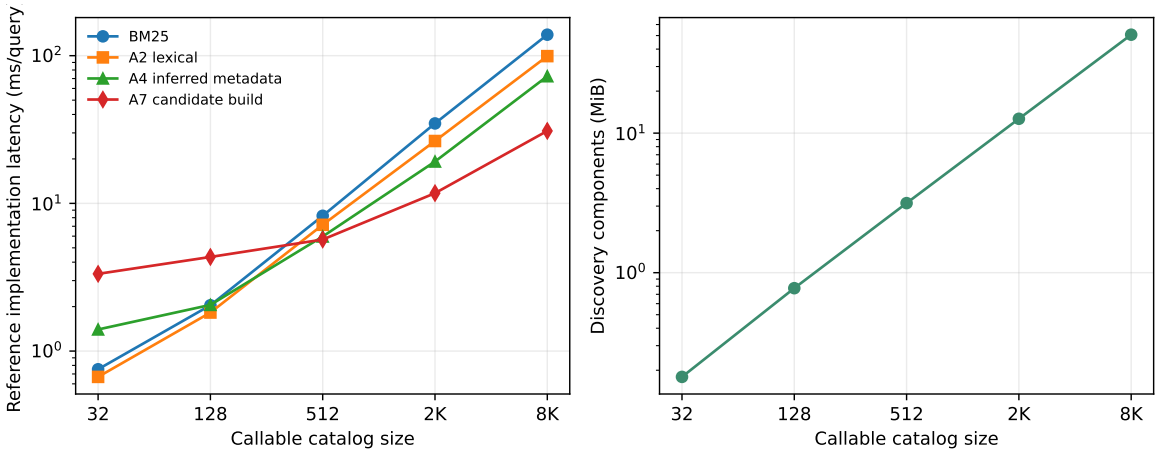

Figure 28: Reference implementation latency and measured discovery-component memory. Both axes are logarithmic; query encoding is batched and reported separately in the machine-readable cost rows.

The large-catalog result reopens A7 only as a retrieval research direction. It does not reopen the default-union or generative JIT gate because recall does not dominate useful precision, safety exposure, and schema context jointly. A production follow-up should use indexed candidate narrowing, calibrate abstention at each catalog scale, and test whether a second-stage reranker can retain the union's recall before any schema becomes model-visible.

J.5 Selection authority and atomic tool records

M8 separates selection from materialization. The agent resolver owns catalog semantics, execution state, tenant and safety filters, and the candidate budget. It emits an authoritative `TOOL_CATALOG_SLICE` whose children are atomic `TOOL_DEFINITION` records. PRA preserves child identity, version, fingerprint, parent relation, field boundaries, and channel provenance, then materializes every admitted child in full. It does not rerank or prune chunks inside a selected schema.

If the slice exceeds the declared budget, the runtime must request a narrower slice, return an error, use an explicitly allowed temporary expansion, or drop whole records in candidate-priority order. Silent partial-schema fallback is invalid. The same substrate can construct tool responses, logs, terminal outputs, database results, and RAG chunks. M9 adds only predefined selection and full views for capability records; dynamic field reduction and detail expansion remain future work rather than hidden behavior in this paper.

E4 reruns the three frozen workflows across five prompt presentations with Top-1 JIT, diversity-union JIT at $K = 2, 4, 6, 8$, static required-set and graph sets, and all safe tools. The frozen Qwen3-0.6B Q4 model obtains 0.000 strict task success at Top-1 and 0.333, 0.333, 0.400, and 0.600 at the four union budgets. All-tools reaches only 0.133 despite complete candidate recall. The prompt presentations are robustness conditions, not training seeds; no model parameter changes. Figure 29 and Table 14 report execution success against disclosure cost rather than treating set recall as execution success.

Policy	Success	Required recall	Schema tokens	Disclosed tools
Top-1 JIT	0.000	0.857	191	2.3
Union JIT, $K = 2$	0.333	1.000	495	5.3
Union JIT, $K = 4$	0.333	1.000	964	6.9
Union JIT, $K = 6$	0.400	1.000	1371	9.2
Union JIT, $K = 8$	0.600	1.000	1922	11.8
Static required-set oracle	0.267	1.000	839	4.0
Static graph set	0.133	1.000	1337	8.0
All tools	0.133	1.000	2411	17.0

Table 14: E4 aggregate over 15 workflows per policy. Schema tokens are cumulative over attempted JIT steps; disclosed tools count unique definitions per workflow.

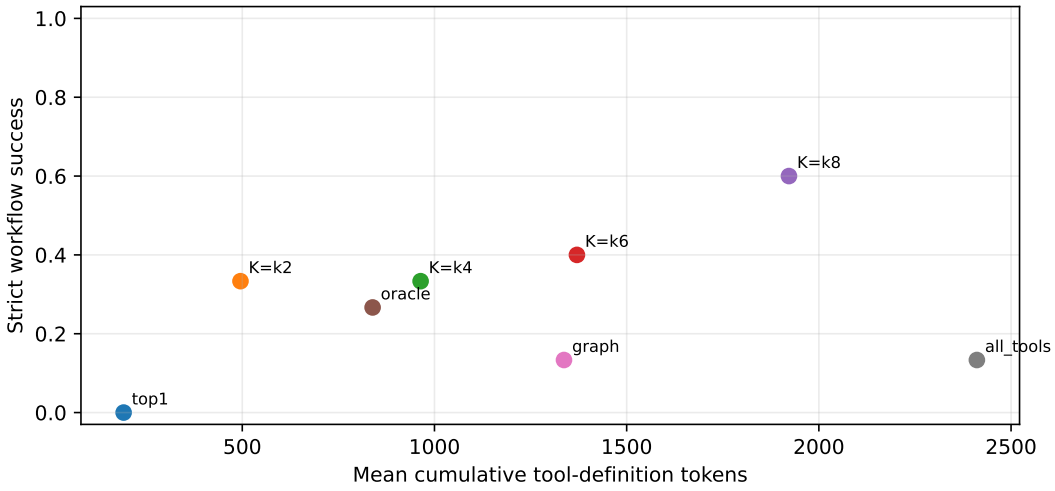

Figure 29: E4 strict workflow success against cumulative visible schema tokens. The frozen model, host authorization, and pure handlers are unchanged across candidate policies.

E5–E6 compare provider-structured full records, flat textual serialization, parameters without a description, descriptions without parameters, and a schema with one required field deliberately removed. Across 20 calls per condition, full atomic records obtain 1.000 accepted execution. Parameter-bearing structured records also reach 1.000 on these simple single-step calls, while description-only records reach 0.150. An explicit content-only internal-chunk router, which selects either the description or parameter-schema segment, likewise reaches only 0.150. Dropping one required field lowers acceptance to 0.000 and causes required-argument omission at rate 1.000. These controls support full selected- tool materialization as the default. M9 retains that atomic full view after selection; it does not introduce progressive detail expansion inside a schema.

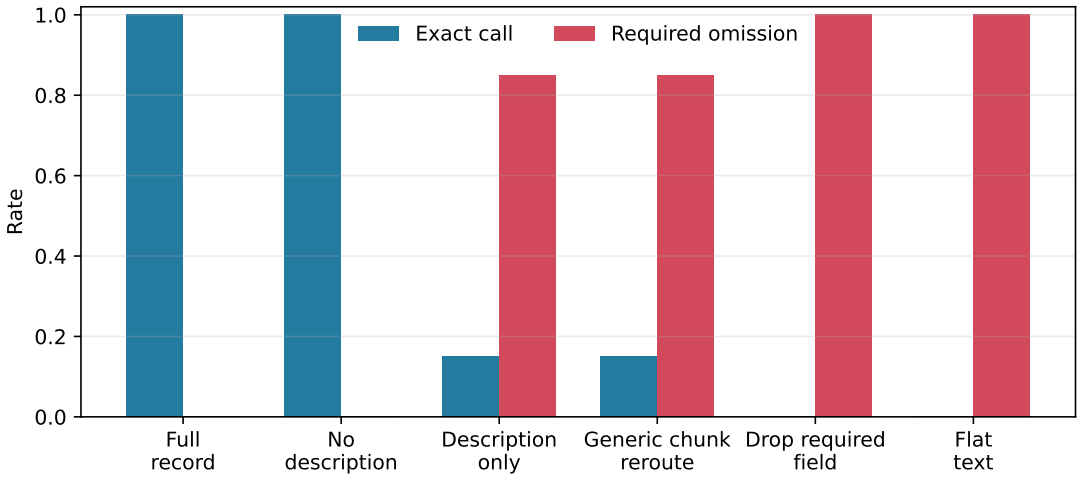

Figure 30: E5–E6 exact calls and required-argument omissions. The generic router selects one internal schema segment; all partial-schema conditions are experimental controls, not runtime fallback policies.

K M9: Typed Progressive Disclosure

Tool and skill records require different detail during choice and use. A model can often choose between capabilities from identity, signature or applicability, and a short description. It needs the complete parameter schema only when constructing a call, and the complete instruction body only when applying a skill. M9 represents this distinction as two deterministic views of one typed, versioned record:

$$R_{\text{select}} \longrightarrow R_{\text{full}}.$$

This transition is not token truncation or learned summarization. Field selection is defined by record type, and both views remain semantic record units. A full view is admitted atomically or not at all.

K.1 Records and explicit two-phase execution

A tool selection view contains stable name, call signature, short description, and side-effect class. Its full view contains the complete nested input schema, required and optional fields, enums and defaults, return schema, identity, version, and authorization-relevant metadata. A declarative skill selection view contains name, description, and when-to-use text. Its full view additionally contains ordered instructions, constraints, examples, dependencies, references, namespace, tenant, and version metadata. M9 supports text-only skills; it rejects script or executable payload metadata.

The runtime performs two ordinary model invocations. Phase A materializes the selection view of every externally bounded candidate and requests one structured capability identity. A validated `CapabilityTransition` permits only `selection→full`. Phase B removes non-selected candidates, materializes the selected full record, and asks the model to produce arguments or follow the selected instructions. Discovery and materialization do not grant execution authority; the existing host validator still checks selected identity, schema, side-effect policy, and request-scoped authorization.

The implementation is deliberately simple. Named views and policy validation live in `src/pra_hf/context_records.py`; the transition and token/KV accounting live in `src/pra_hf/progressive_disclosure.py`; and declarative skill records live in `src/pra_hf/skill_records.py`. There is no generation callback, decoder interruption, confidence-triggered expansion, arbitrary compression, or mid-token materialization. Those dynamic mechanisms belong to a later large-record study.

For candidate full-view tokens F_K, Phase-A selection tokens A_K, and the selected full-view tokens B, we report

$$\text{DisclosureRatio} = \frac{A_K}{F_K}, \qquad \text{TotalDisclosureRatio} = \frac{A_K + B}{F_K}.$$

The latter counts both model phases and therefore does not hide the second invocation.

K.2 Protocol

The tool experiment freezes 8 semantic-hard requests over the 18 M8 callables. The skill benchmark contains 50 realistic, semantically confusable declarative skills and 50 name-blind requests split evenly between validation and test. Validation selects the same external BM25, automatic semantic, compact-embedding, and fused discovery stack used for tools. Generation uses 4 held-out skill requests. Both experiments compare full-all (T0/S0), selection-only (T1/S1), selection-to-full (T2/S2), and oracle-full (T3/S3). Tool budgets retain the original bounded-palette grid; skills use $K \in \{2, 4, 8, 16, 24, 32\}$ over the 50-record catalog.

All generation rows use the exact frozen Qwen3-0.6B revision in FP16 on an NVIDIA GeForce GTX 950M, deterministic decoding, and no parameter updates. The small eight-request generation cohorts are mechanism checks, not population estimates. Tool evaluation separates required Recall@K, capability choice, schema validity, argument semantics, host acceptance, and strict success. Skill evaluation uses deterministic identity markers, required output headings, and forbidden external-action claims. Timing includes CUDA synchronization, serialization, both TTFTs, and both generations.

K.3 Discovery and view size

The shared discovery stack transfers without a second skill-specific retrieval system. On the 50-query skill test split, BM25 obtains 0.600 Top-1, compact embeddings obtain 0.800, and validation-selected fusion obtains 0.680. Fused Recall@K rises from 0.840 at $K = 2$ to 0.940 at $K = 4$, 0.960 at $K = 8$, and 1.000 at $K = 16$. The corresponding tool bars in Figure 31 reuse M8's 144 test requests and should not be read as a paired cross-type comparison.

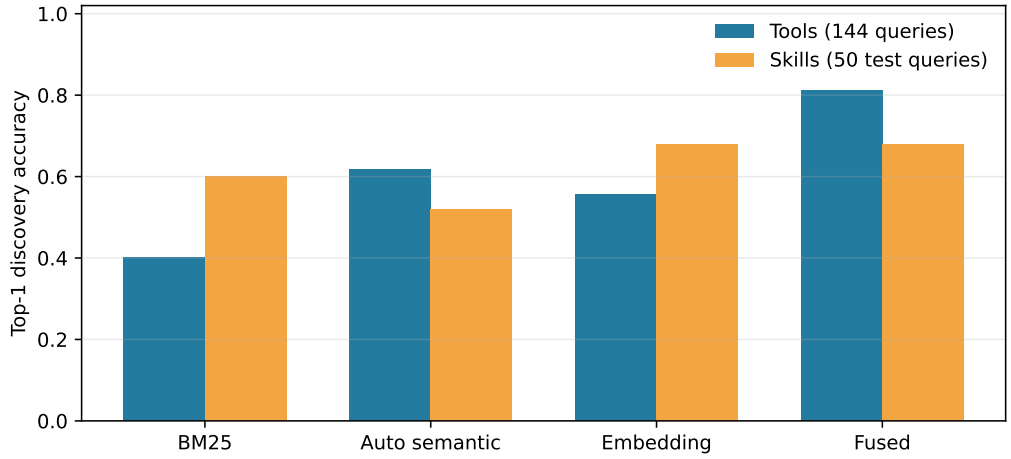

Figure 31: The same external discovery families applied to tools and declarative skills. Cohort sizes differ; the figure tests substrate reuse, not which resource type is intrinsically easier.

Full tool schemas contain a median 233.5 tokens (p95 244), and their selection views retain a median 0.368 of that count. Full skill instructions contain a median 685 tokens (p95 1,353), while skill selection views retain 0.150. These bodies are intentionally nontrivial: full-all instruction cost therefore grows linearly with the candidate palette.

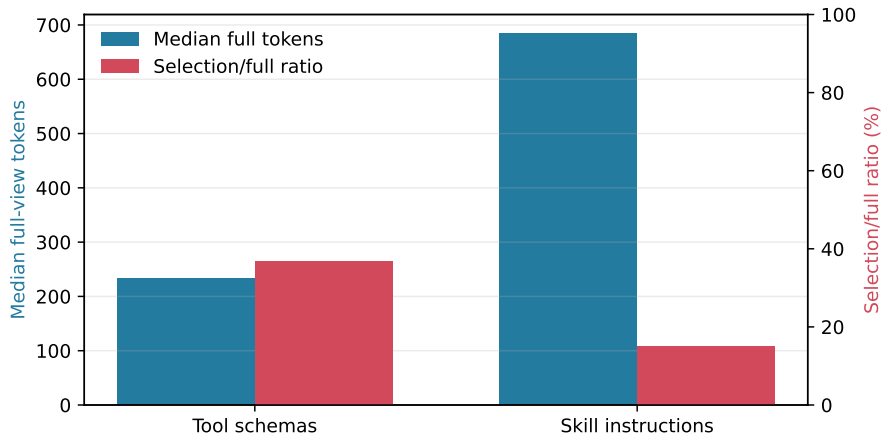

Figure 32: Median full-view tokens and median selection/full token ratio for the 18 tool schemas and 50 declarative skills. Ratios are percentages in the red bars; no view is formed by first-N-token truncation.

Bucket	n	Instr.	Select	Full	Saved	R@8	Choice	Success
Short	17	177	103	510	0.78	0.88	0.00	0.00
Medium	17	345	103	687	0.78	1.00	0.00	0.00
Long	16	972	104	1341	0.70	1.00	1.00	1.00

Table 15: Skill results by measured instruction-length bucket. Token columns are means; "Saved" is the mean $K = 8$ progressive token saving. Choice and success come from four stratified generation cases (two short, one medium, one long), so they are mechanism diagnostics rather than bucket-level estimates.

K.4 Tools: choice improves at moderate budgets, not at catalog scale

Table 16 reports the full tool matrix. At $K = 4$, T2 raises choice accuracy from 0.500 to 0.875 and strict success from 0.250 to 0.375, while exposing 0.589 of T0's capability tokens. At $K = 8$, T0 and T2 both obtain 0.625 choice accuracy; T2 uses 0.479 of full-all tokens but adds 0.492 seconds mean wall time. At all 18 tools, T2 choice falls to 0.375 versus 0.625 for T0. Compact views do not remove the need for bounded discovery.

Selection-only obtains no strict successes: compact signatures are sufficient for choosing in some rows, but are not a reliable replacement for full provider schemas during argument construction. T3 obtains 1.000 schema validity and host acceptance at every budget, yet only 0.375 strict success because five of eight requests elicit non-exact argument values even when the correct tool is supplied alone. This oracle ceiling is why M9 does not interpret low strict success as a routing-only failure.

K.5 Skills: context economy under a bounded candidate set

At $K = 8$, the full-all condition does not fit the declared 8,192-token runtime limit for any of the four stratified generation cases. Selection-to-full fits all four while exposing 0.300 of full-all capability tokens. It chooses and applies the correct skill in one case (0.250 strict success); oracle-full obtains 1.000. S1 fails the required identity and output contract throughout, confirming that compact

K	n	Condition	Recall	Choice	Valid/follow	Success	Tokens	Wall (s)
2	8	T0 full-all	0.88	0.75	0.88	0.50	472.00	4.36
2	8	T1 selection-only	0.88	0.00	0.00	0.00	173.88	2.93
2	8	T2 select→full	0.88	0.75	0.88	0.25	381.88	5.00
2	8	T3 oracle-full	0.88	1.00	1.00	0.38	236.88	4.04
4	8	T0 full-all	1.00	0.50	0.88	0.25	945.50	4.63
4	8	T1 selection-only	1.00	0.00	0.00	0.00	349.12	2.88
4	8	T2 select→full	1.00	0.88	0.88	0.38	557.12	5.43
4	8	T3 oracle-full	1.00	1.00	1.00	0.38	236.88	4.06
6	8	T0 full-all	1.00	0.62	1.00	0.25	1417.00	4.96
6	8	T1 selection-only	1.00	0.00	0.00	0.00	523.62	3.31
6	8	T2 select→full	1.00	0.75	1.00	0.25	759.88	6.42
6	8	T3 oracle-full	1.00	1.00	1.00	0.38	236.88	4.08
8	8	T0 full-all	1.00	0.62	0.88	0.38	1889.25	5.75
8	8	T1 selection-only	1.00	0.00	0.00	0.00	697.88	3.44
8	8	T2 select→full	1.00	0.62	0.88	0.12	904.50	6.25
8	8	T3 oracle-full	1.00	1.00	1.00	0.38	236.88	3.92
18	8	T0 full-all	1.00	0.62	0.88	0.38	4235.00	8.49
18	8	T1 selection-only	1.00	0.00	0.00	0.00	1567.00	8.55
18	8	T2 select→full	1.00	0.38	0.62	0.00	1715.12	8.85
18	8	T3 oracle-full	1.00	1.00	1.00	0.38	236.88	4.06

Table 16: Tool progressive-disclosure matrix. n is the number of context-fitting generation cases. "Valid/follow" is schema-valid call rate. Strict success also requires the exact benchmark arguments and accepted host execution.

applicability text is a choice surface rather than a replacement for the authoritative instructions. The four-case cohort is a mechanism check: it establishes feasibility and exposes the remaining choice error, but it is too small for a population-level quality claim.

K	n	Condition	Recall	Choice	Valid/follow	Success	Tokens	Wall (s)
2	4	S0 full-all	0.50	0.25	1.00	0.25	1764.00	20.83
2	4	S1 selection-only	0.50	0.00	0.00	0.00	205.50	8.21
2	4	S2 select→full	0.50	0.00	0.50	0.00	1001.50	12.45
2	4	S3 oracle-full	0.50	1.00	1.00	1.00	757.25	11.37
4	2	S0 full-all	0.50	0.00	1.00	0.00	2796.50	32.11
4	4	S1 selection-only	0.50	0.00	0.00	0.00	409.75	6.23
4	4	S2 select→full	0.50	0.25	0.75	0.25	1372.00	14.54
4	4	S3 oracle-full	0.50	1.00	1.00	1.00	757.25	11.46
8	0	S0 full-all	0.50	–	–	–	–	–
8	4	S1 selection-only	0.50	0.00	0.00	0.00	833.25	9.17
8	4	S2 select→full	0.50	0.25	0.75	0.25	2006.25	17.85
8	4	S3 oracle-full	0.50	1.00	1.00	1.00	757.25	12.41
16	0	S0 full-all	1.00	–	–	–	–	–
16	4	S1 selection-only	1.00	0.00	0.00	0.00	1660.50	30.30
16	4	S2 select→full	1.00	0.25	0.75	0.25	2836.75	21.93
16	4	S3 oracle-full	1.00	1.00	1.00	1.00	757.25	13.03
24	0	S0 full-all	1.00	–	–	–	–	–
24	4	S1 selection-only	1.00	0.00	0.00	0.00	2493.50	25.56
24	4	S2 select→full	1.00	0.25	0.75	0.25	3669.75	34.40
24	4	S3 oracle-full	1.00	1.00	1.00	1.00	757.25	15.08
32	0	S0 full-all	1.00	–	–	–	–	–
32	3	S1 selection-only	1.00	0.00	0.00	0.00	3318.00	35.66
32	3	S2 select→full	1.00	0.33	0.67	0.33	4657.67	39.31
32	3	S3 oracle-full	1.00	1.00	1.00	1.00	840.00	11.50

Table 17: Declarative-skill progressive-disclosure matrix. "Valid/follow" is deterministic instruction-contract success. Quality n counts rows that fit the runtime envelope. Full-all has no fitting rows from $K = 8$ onward. Three $K = 32$ rows per executable condition completed; the fourth stalled during the long selection prompt and is reported as missing, not imputed.

At $K = 32$, full-all would expose 26,094 mean capability tokens and cannot run under the measured GPU limit. Selection-to-full exposes 4,658 tokens in the three completed rows and obtains 0.333 strict success. Because one stratified case is absent, the $K = 32$ value is descriptive only. This stress result establishes feasibility under one hardware envelope; it does not imply that larger-memory runtimes cannot execute full-all.

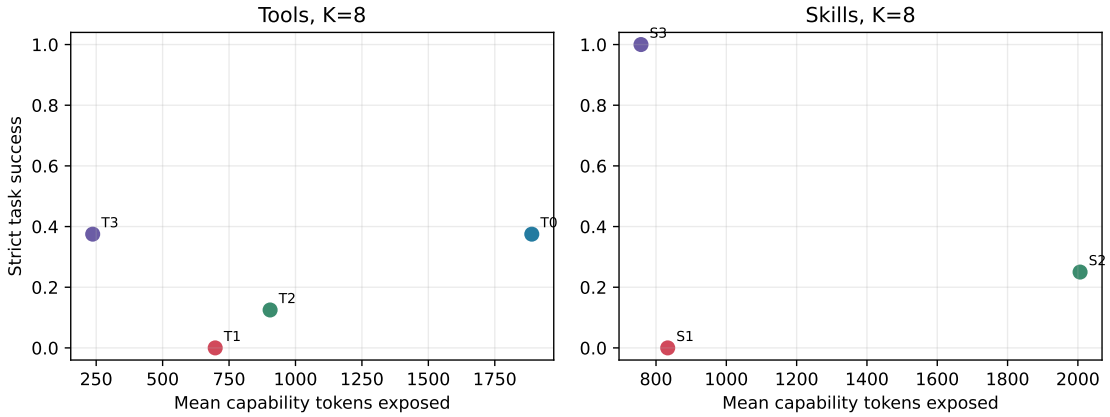

Figure 33: Strict task success against total capability tokens exposed at $K = 8$. Totals include both phases for T2/S2. The oracle points isolate the call or instruction-following ceiling after externally correct selection.

K.6 Gate decision

The architectural gate opens: typed selection/full views, explicit transitions, atomic selected records, and shared tool/skill discovery work without callback machinery. The skill inclusion gate also opens because fused discovery exceeds BM25 by 0.160 Top-1 and selected-full execution cuts instruction context while remaining deterministically evaluable. The deployment gate is conditional. Selection-to-full is the SDK policy for an already bounded capability palette, but the resolver must retain its calibrated candidate limit and host safety checks. This small cohort does not establish statistical non-inferiority, and the all-tool result rejects using compact views as a substitute for discovery.

L M10: Large-Catalog Palette Use

The 8,192-tool retrieval result leaves a downstream question open. A larger candidate set is useful only when the model can identify the target among its additional confusable tools. M10 freezes the same callable generator and eight M9 tool requests, then separates three probabilities:

$$P(t \in C), \qquad P(\hat{t} = t \mid t \in C), \qquad P(\hat{t} = t).$$

The first is resolver recall, the second is model discrimination conditional on a successful resolver, and the third is their end-to-end product. Every raw row retains the chosen identity, candidate count, channel sources and ranks, unsafe and useful candidates, view tokens, and wall time.

L.1 Protocol and model boundary

The base retrieval grid covers $N \in \{32, 128, 512, 2048, 8192\}$, $K \in \{2, 4, 6, 8, 10\}$, and BM25, frozen fusion, raw union, and diversity union. Seed 11 covers the Cartesian grid. The decisive $N = 8192, K = 10$ comparison uses all five catalog seeds and 40 query–seed pairs; a fixed agreement bonus is an additional control. A targeted diversity-only extension evaluates $K \in \{12, 16, 24, 32\}$ on those same 40 pairs. Target-absent palettes are recorded without calling the chooser, because their end-to-end outcome is already determined; target-present palettes use the identical frozen chooser. Candidate identities are bound to labels and the official Ollama `qwen3:0.6b` Q4 artifact

generates one label at temperature zero. This low-memory artifact is kept separate from the exact FP16 CUDA model used below for call construction. Catalog seeds vary generated distractors, not model weights.

The 40-row recall values in Table 18 differ from the 144-query values in Table 13 because M10 uses the smaller argument-annotated generation cohort. They answer different questions: M8 estimates resolver behavior on all frozen test requests; M10 tests whether a model can use selected palettes on requests with executable expected calls.

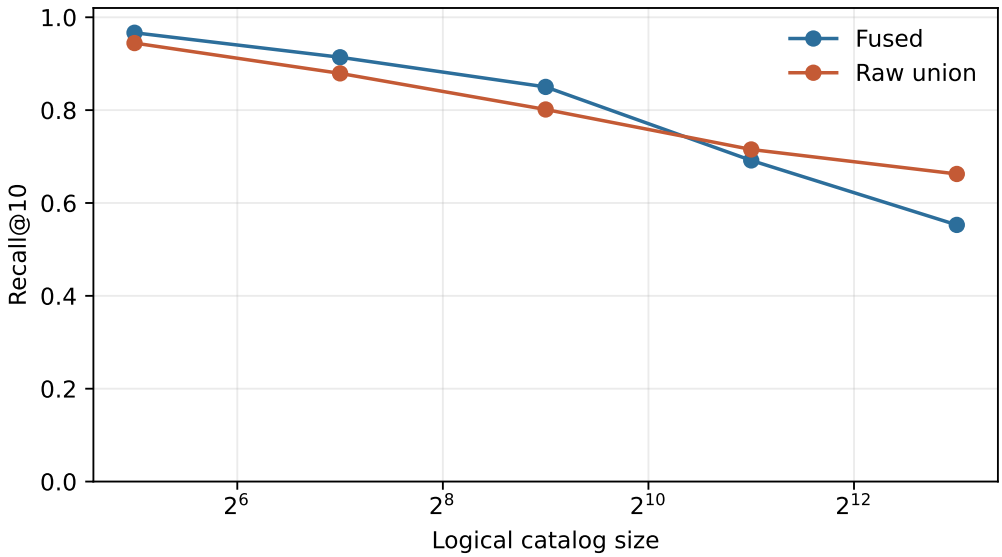

Figure 34: Five-seed large-catalog Recall@10. Union increasingly helps candidate generation as the logical catalog grows, before any candidate is shown to the generation model.

L.2 Candidate recall is not candidate use

Policy	n	Recall	Cond. choice	End-to-end	Unsafe choice	Tokens
BM25	40	0.375	0.600	0.225	0.000	1465
Fused	40	0.300	0.083	0.025	0.075	1514
Raw union	40	0.375	0.000	0.000	0.000	1464
Diversity union	40	0.375	0.667	0.250	0.000	1465
Agreement union	40	0.375	0.467	0.175	0.050	1464

Table 18: Generated capability choice at $N = 8192, K = 10$ across eight requests and five catalog seeds. Conditional choice excludes target-absent rows; end-to-end choice counts them as failures. Tokens expose compact selection views only.

Raw union raises recall from 0.300 for frozen fusion to 0.375, but Qwen chooses none of its 15 available targets. The model cannot use raw union's additional hypotheses. Diversity union retains the same 0.375 recall and chooses 10/15 available targets, yielding 0.250 end-to-end choice. Fusion obtains 1/12 and 0.025 respectively. On paired query–seed identities, diversity wins ten rows that fusion loses and loses one row that fusion wins (exact two-sided $p = 0.0117$). The fixed agreement

bonus obtains 7/15 conditional successes; its paired effect versus fusion is unresolved ($p = 0.0703$). This is evidence for candidate ordering and channel diversity, not for broad schema disclosure.

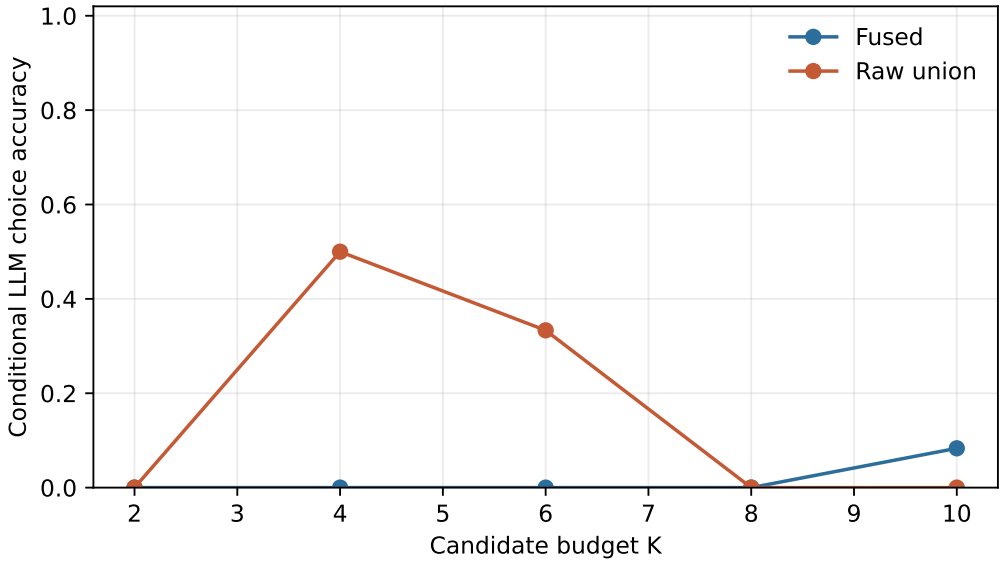

Figure 35: Conditional model choice across candidate budgets. A resolver gain becomes operational only when the model can distinguish target from admitted distractors.

The compact/full view control uses 128 matched decisions over four stratified requests. Compact views average 0.453 of full-view tokens and obtain 10/128 correct choices, versus 7/128 for full views. Both rates are low and this study does not claim non-inferiority. It shows that exposing every nested schema field does not repair large-catalog selection for this small model. Three executable large-schema controls contain nested dataclasses, lists, optional objects, enums, literals, and multiple required fields; their full views contain a median 721 tokens (maximum 736), while selection views retain 0.219–0.279.

L.3 Selection to atomic execution

The causal execution matrix returns to the exact frozen Qwen3-0.6B FP16 revision. At $N = 8192$ and $K \in \{4, 8\}$, all five catalog seeds and four requests compare full-all, compact-only, compact-to-full, and oracle-to-full. The model first selects from complete compact records. The runtime then removes losing records, materializes one full schema atomically, asks for arguments, and submits the result to the same host validator. No resolver score grants execution authority.

Policy	View	Recall	Choice	Accepted	Strict	Tokens
Fused	Full-all	0.300	0.100	0.750	0.100	2615
Fused	Select→full	0.300	0.100	0.650	0.100	1495
Fused	Oracle full	0.300	1.000	1.000	0.750	278
Raw union	Full-all	0.250	0.150	0.700	0.150	2561
Raw union	Select→full	0.250	0.050	0.600	0.050	1441
Raw union	Oracle full	0.250	1.000	1.000	0.750	278
Diversity union	Full-all	0.250	0.000	0.500	0.000	2558
Diversity union	Select→full	0.250	0.250	0.550	0.250	1419
Diversity union	Oracle full	0.250	1.000	1.000	0.750	278

Table 19: Exact-FP16 large-catalog tool execution at $N = 8192, K = 8$ ($n = 20$ per row). Strict success requires target choice, exact benchmark arguments, and accepted host execution. Oracle full isolates post-selection call quality.

Diversity compact-to-full recovers every target-present row (5/5), obtains 5/20 strict accepted calls, and exposes 1,419 capability tokens on average, 0.556 of its 2,558-token full-all palette. It materializes one full schema and avoids 2,294 full-schema tokens. Measured native-K/V accounting falls from 293.4 MB-equivalent bytes for full-all to 162.7 MB-equivalent bytes. Raw union obtains 1/20 strict calls and fused obtains 2/20. Diversity versus fusion has the same positive direction in 4 versus 1 discordant rows, but the 20-pair effect is unresolved (exact $p = 0.375$). Oracle-to-full obtains 15/20 strict calls, locating the remaining ceiling in discovery and model choice rather than schema execution alone.

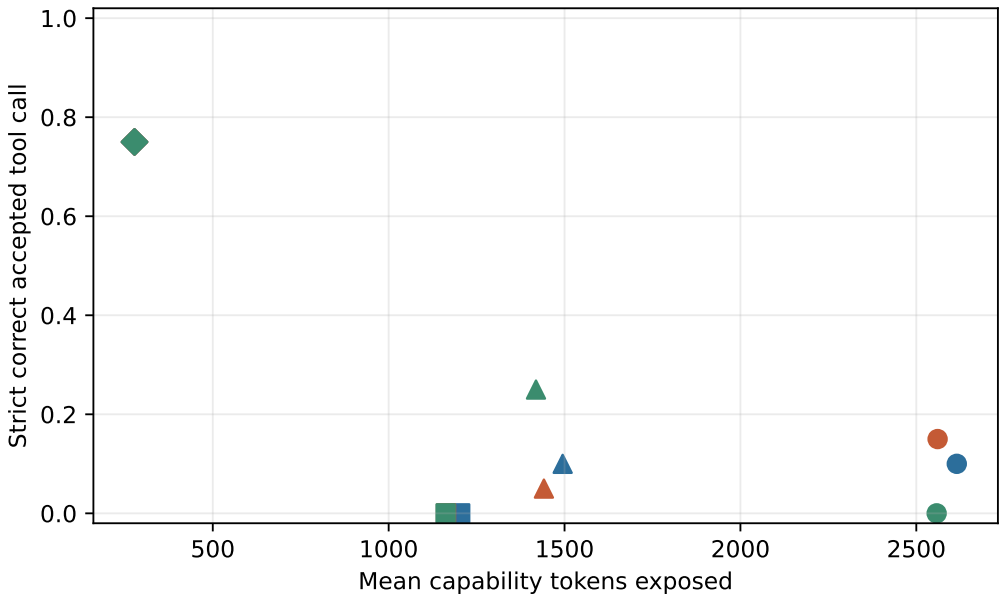

Figure 36: Large-catalog strict accepted calls against total capability tokens at $N = 8192, K = 8$. Two-phase totals include compact selection views and the one selected full schema.

L.4 Large-catalog JIT remains recall-bound

Two two-step workflows pair account retrieval with update and document search with export. For each step the resolver searches the full 8,192-tool logical catalog, exposes $K \in \{4, 8\}$ compact records, materializes one selected full schema, validates the call, records a typed observation, and advances. The five catalog seeds yield ten workflows per policy.

Policy	Steps	Recall	Cond. choice	Accepted	Workflow	Tokens
Fused	20	0.300	0.333	0.650	0.000	1495
Raw union	20	0.250	0.200	0.600	0.000	1441
Diversity union	20	0.250	1.000	0.550	0.000	1419

Table 20: Large-catalog two-step JIT at $K = 8$. Conditional choice can be high while workflow success remains zero when a required step is absent from the palette.

Diversity union again uses its successful palettes well: it chooses 5/5 target-present steps. But per-step recall is only 0.250, so no policy completes either required step in all ten workflows. Raw union and fusion also obtain zero workflow success. No generated proposal is an unsafe tool, but host acceptance remains 0.55–0.65 because wrong identities and malformed or incomplete arguments are still rejected. The default-union gate therefore remains closed. The supported policy is frozen fusion by default, with diversity union exposed as an experimental high-recall mode and explicit host authorization after every selection.

M Deferred External Validations

Work outside this paper returns to the broader persistent-context thesis: hierarchical skills, mutation and revocation, bounded `#_head` history, superseded facts, and permission-scoped child sessions

$$M_{\text{child}} = R(M_{\text{parent}}) \cup \Delta M_{\text{child}}.$$

Cross-model replication is also required because M2–M10 use one Qwen model family for pretrained mechanisms and downstream capability choice. The generated large-catalog study does not replace natural catalogs or a tool-identity-disjoint benchmark; both should test whether the observed lexical advantage and union recovery survive distribution shift. The OpenAI-compatible endpoint belongs after these mechanism checks, so harness behavior does not hide resolver, disclosure, or execution failures.

N Reproducibility

M8's E1–E3 driver is `run_auto_union_records.py`. Candidate sets for E4 are frozen by `run_union_jit_records.py` with `--prepare-only`. The five-presentation E4–E6 driver is `run_union_jit_ollama.py`. It uses the official `qwen3:0.6b` Q4_K_M artifact on CPU with temperature zero, no GPU offload, per-call unloading, and deterministic host-side schema validation. This low-memory runtime is separate from the exact-FP16 M2–M7 rows. The runner checkpoints at each completed condition. The companion summarizer regenerates tables, macros, findings, and all four M8 figures.

The zero-configuration source, type, quality, and A0–A7 channel ablations use `run_auto_discovery_ablation.py`; `summarize_auto_discovery_ablation.py` regenerates its table,

macros, four figures, and machine-readable summary. The driver constructs one frozen record for each of the same 18 callables, selects embedding representation and fusion weights only on validation, and evaluates all reported policies on 144 test requests. It used the compact BGE encoder on an NVIDIA GeForce GTX 950M and completed in 103 seconds. The manifest records every automatic field's source. The A7 JIT status is a checked-in gate-closed artifact because no union condition met the validation recall-and-safety criterion.

The semantically confusable scale follow-up uses `run_scaled_auto_discovery.py`; it checkpoints one gzip shard per catalog size and seed and resumes only when the protocol hash matches. The run covers five sizes, seeds $\{11, 23, 37, 53, 71\}$, 144 frozen test requests, three ranking policies, six matched candidate strategies, and eight budgets on CUDA. `summarize_scaled_auto_discovery.py` audits row completeness and budget compliance, aggregates catalog-seed means and standard deviations, and regenerates the table, macros, four figures, and machine-readable findings. The raw rows distinguish index construction, query encoding, channel scoring, candidate construction, component memory, logical schema tokens, and selected schema tokens.

M9 is prepared by `prepare_progressive_disclosure.py`, executed by `run_progressive_disclosure.py`, and summarized by `summarize_progressive_disclosure.py`. The preparation stage freezes selection/full views, the 50-skill catalog, 100 name-blind skill queries, validation-selected fusion weights, candidate sets, and disclosure costs. The generation runner checkpoints each condition and records prompt/context fit, native-K/V bytes, both TTFTs, transition overhead, wall time, raw output, and deterministic evaluator fields. The summarizer regenerates two full condition tables, three figures, macros, and machine-readable findings. No model or encoder weights are committed.

M10 is prepared by `prepare_missing_experiments.py`, executed by `run_missing_experiments.py`, and summarized by `summarize_missing_experiments.py`. Preparation regenerates the same five callable catalogs, freezes 5,000 policy/budget palettes, serializes 3,122 unique candidate views, and records three realistic nested-schema stress tools. The choice phase uses the official Ollama `qwen3:0.6b` Q4 artifact with persistent CPU loading and checkpoints every bounded block. The execution phase resumes the same checkpoint with the exact FP16 local revision on CUDA. Raw artifacts contain 968 compact choices, 160 matched full-view choices, 704 execution rows, 120 JIT step rows, per-seed identities, paired exact effects, timing, token/KV accounting, and generated figures and tables. The split precision protocol is explicit in the manifests; Q4 and FP16 rows are never pooled as repeated seeds of one estimator.

The M0 benchmark entry points are `experiments/paper6_5_tools/run_m0_policy_study.py` and `summarize_m0_policy_study.py`. Machine-readable outputs include the manifest, one row per policy/query trace, index costs, seed summaries, stratum summaries, oracle distributions, and plots under `docs/papers/shared/results/paper6_5_tools`. The recorded environment is Python 3.10.11 on Windows, Intel64 Family 6 Model 94. All reported M0 results are deterministic conditional on the five catalog seeds.

M1 is reproduced by `run_m1_toy_model.py` followed by `summarize_m1_toy_model.py`. Checked-in artifacts include all 720 evaluation rows, per-step training histories, seed and aggregate summaries, paired effects, a manifest, and generated figures. The run used PyTorch on an NVIDIA GeForce GTX 950M and seeds $\{11, 23, 37, 53, 71\}$. Exact configuration and runtime metadata are recorded in `paper6_5_tools/m1/m1_manifest.json`.

M2–M4 are reproduced by `run_m2_m4_pretrained.py`. M5 uses `run_m5_disclosure.py`; its direct, reactive, and oracle model rows reuse the frozen M4 summaries, while P6/P8 are newly executed static disclosures. M6 uses `run_m6_native_discovery.py`. The final tables, macros, and figures are generated by `summarize_m2_m6.py`. Raw rows contain one call, workflow step, policy/task, or routing-query record as appropriate. Manifests record the frozen model revision,

CUDA device, presentation seeds, index accounting, checkpoint provenance, and the fact that all execution handlers are pure in-memory fixtures. M6.5 uses `run_m6_5_external_semantics.py`; M7 uses `run_m7_semantic_native.py`; and `summarize_semantic_hard.py` regenerates paired effects, tables, macros, and figures. Their manifests pin query fingerprints, split discipline, encoder/model revisions, representation selection, thresholds, and CUDA runtime. Tool vectors are committed, but third-party encoder weights and raw native Q/K are not.

O Experimental Archive Summary

A large agent catalog should not be mistaken for the context needed on every turn. M0 resolves typed identities without exposing the catalog, and M1 shows that one selected native-K/V definition is used causally. M2–M4 carry the mechanism into a frozen pretrained model: selected schemas support exact calls, host authorization remains separate from generation, typed observations retain provenance, and reactive JIT supports bounded multi-step execution better than static disclosure in this cohort.

The extensions also locate two boundaries. M5 capability graphs improve set recall but not task success; more useful tools visible at once are still more actions for a small model to sequence. M6's canonical queries favor lexical indexing, whereas M6.5 shows that authored tags, dictionaries, and compact embeddings recover semantic-hard intent that BM25 misses. M7 then finds no zero-shot native benefit on those same hard rows. These are positive external- resolver results and bounded native stop results, not evidence that all tool semantics are lexical or that trained native geometry cannot work.

The supported architecture combines explicit identity, indexed lexical search, provenance-bearing dictionaries, compact embedding fallback, calibrated ASK, reactive disclosure, host-authorized execution, and typed observations. It works with an opaque generation API; graph neighborhoods and native Q/K remain optional research interfaces. M8 adds ordinary callables as typed records and atomic PRA transport of the selected palette. On this catalog, generic synonyms supply most of the isolated gain, and fusion with automatic tags and embeddings matches the rich manual result. Union remains optional because it does not dominate fusion on the original cohort. Under five semantically confusable catalog seeds, frozen hybrid ranking degrades faster than lexical evidence; bounded union restores part of the 8K recall but leaves low useful precision and nonzero unsafe exposure. The result motivates indexed high-recall retrieval followed by calibrated reranking, not default broad disclosure.

M9 narrows the cost after discovery. Deterministic selection views preserve a bounded choice palette; the runtime then removes losing candidates and exposes one complete authoritative tool schema or declarative skill. At $K = 8$, skill selection-to-full remains executable at 0.300 of full-all capability tokens, whereas full-all no longer fits. Its 0.250 strict success and 1.000 oracle ceiling localize the remaining error to selection rather than instruction availability. The tool result is less uniform and degrades at all-catalog scale, so progressive disclosure complements rather than replaces calibrated discovery. Basic declarative skills are now covered; hierarchical skills, persistent history, mutation, revocation, and session inheritance remain untested.

M10 closes the downstream large-catalog question without erasing its boundary. Raw union's recall gain is unusable by the Q4 chooser, whereas diversity union turns 10/15 target-present palettes into correct choices and significantly improves paired end-to-end choice over frozen fusion. In exact-FP16 execution, diversity selection-to-full converts all 5/5 target-present rows into strict accepted calls while using 0.556 of full-all capability tokens. The two-step JIT result nevertheless remains zero because only one quarter of required steps contain the target. Typed progressive

disclosure therefore solves the cost of using a bounded palette; it does not solve large-catalog discovery. The diversity-only high-K extension also rules out a simple breadth remedy on the executable cohort: target recall remains 0.375 through $K = 32$, while conditional choice declines from 0.667 at $K = 10$ to 0.400. Larger compact palettes are affordable, but they are not free of decision interference. Fusion remains the SDK default, diversity union is a measured experimental high-recall mode, and callbacks or dynamic detail generation remain outside this paper.

References

[1] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, et al. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems*, 2023.

[2] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large Language Model Connected with Massive APIs. In *Advances in Neural Information Processing Systems*, 2024.

[3] Yujia Qin, Shihao Liang, Yining Ye, et al. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-World APIs. In *International Conference on Learning Representations*, 2024.

[4] Vladimir Karpukhin, Barlas Oguz, Sewon Min, et al. Dense Passage Retrieval for Open-Domain Question Answering. In *Proceedings of EMNLP*, pages 6769–6781, 2020.

[5] Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, 2020.

[6] Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. Quest: Query-Aware Sparsity for Efficient Long-Context LLM Inference. In *International Conference on Machine Learning*, 2024.

[7] Anthropic. Agent Skills public repository and skill-creator specification. `https://github.com/anthropics/skills`, accessed August 2026.

[8] Zhengliang Shi, Yuhan Wang, Lingyong Yan, et al. Retrieval Models Aren't Tool-Savvy: Benchmarking Tool Retrieval for Large Language Models. arXiv:2503.01763, 2025.

[9] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models. In *Proceedings of EMNLP*, 2023.

[10] Headroom Labs. Reversible Compression (CCR): Compress–Cache–Retrieve architecture. `https://docs.headroomlabs.ai/docs/ccr`, accessed August 2026.

[11] Stephen Robertson and Hugo Zaragoza. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

[12] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. In *Proceedings of SIGIR*, 2009.